SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

알고리즘 기본

Stack 1

Python

학습시작

접시를 쌓아두었다가 꺼낼 때의 순서를 떠올려 봅시다.

식당에서 설거지를 끝낸 접시를 쌓는 곳이 있습니다.
아래와 같이 순서대로 접시를 올려놓습니다.

1번 접시 → 2번 접시 → 3번 접시

직원이 접시를 두 개 꺼냈다면, 꺼낸 접시 번호의 순서는?

- 뒤로가기, 실행 취소, 함수 실행 순서 같은 흐름을 구현할 수 있습니다.
- 중첩된 구조(예: 괄호, 블록, 태그 등)를 관리하는 방식에 익숙해질 수 있습니다.
- 데이터를 임시로 저장했다가, 정해진 시점에 꺼내는 구조를 직접 다룰 수 있습니다.

Objectives

학습목표

- 스택 자료구조의 개념과 특징을 설명할 수 있다.
- 스택을 활용한 기본 응용 문제를 해결할 수 있다.
- 재귀호출의 동작 원리와 스택 구조의 관계를 이해한다.
- Memoization 기법을 통해 재귀호출의 성능을 향상시킬 수 있다.
- 동적 계획법(DP)의 개념과 구현 방식을 설명하고 적용할 수 있다.
- 스택을 이용한 깊이 우선 탐색(DFS)을 구현할 수 있다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

Stack 자료구조 이해

- Stack의 개념과 특성
- Stack의 기본 연산
- Stack 구현 실습

Stack 응용

- 괄호 검사
- Function Call

Stack 기반 문제 해결 기법

- 재귀호출
- Memoization
- DP
- DFS

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Stack 자료구조 이해

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Stack의 개념과 특성

스택(Stack)의 개념

스택
Stack

물건을 쌓아 올리듯 자료를 쌓아 올린 형태의 자료구조

대표적인 선형 자료구조 중 하나

스택에 자료를 삽입하거나, 스택에서 자료를 꺼낼 수 있습니다

선형구조 :
자료 간의 관계가 1:1의 관계를 갖는다. (예: 배열, 리스트)

<개념1_알고리즘 기본_Stack 1_선형구조>

비선형구조 :
자료 간의 관계가 1:N의 관계를 갖는다. (예: 트리, 그래프)

<개념2_알고리즘 기본_Stack 1_비선형구조>

스택(Stack)의 특성

후입선출

LIFO, Last-In-First-Out

가장 마지막에 넣은 자료가 가장 먼저 나오는 것

예를 들어, 스택에 1, 2, 3을 차례대로 넣으면 꺼낼 때는 3, 2, 1 순서로 나오게 됩니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Stack의 기본 연산

스택을 프로그램에서 구현하기 위해서 필요한 자료구조와 연산

- 배열을 사용해 구현할 수 있음

- 파이썬에서는 리스트를 사용해 구현할 수 있습니다.

- 저장소 자체를 스택이라 부르기도 함

- 용도에 따라 메모리의 일부를 스택으로 부릅니다.

- 스택에서 마지막 삽입된 원소의 위치

- 스택 포인터, top으로 부르며 데이터를 넣거나 뺄 때 기준이 되는 위치입니다.

자료구조 :
스택은 선형 자료구조이다.

<개념3_알고리즘 기본_Stack 1_자료구조>

배열 :
자료를 선형으로 저장할 저장소

<개념4_알고리즘 기본_Stack 1_배열>

스택의 연산

- 삽입 (Push)

 - 저장소에 자료를 저장하는 연산으로, 보통 push라고 부릅니다.

- 삭제 (Pop)

 - 저장소에서 삽입한 자료의 역순으로 꺼내는 연산으로, 보통 pop이라고 부릅니다.

- 스택이 공백인지 아닌지를 확인하는 연산 (isEmpty)

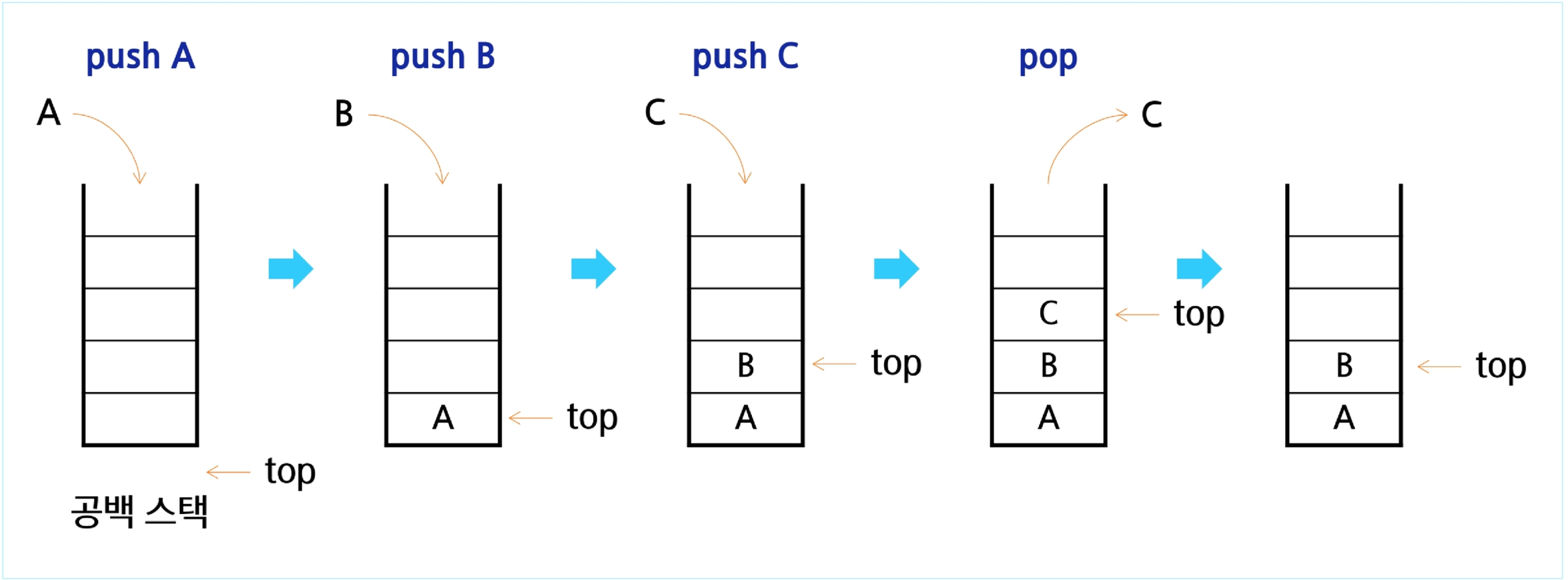
 - 스택이 비어 있으면 True, 아니면 False를 반환합니다.

- 스택의 top에 있는 item(원소)을 반환하는 연산 (peek)

 - 삭제는 하지 않습니다.

스택의 삽입/삭제 과정

- 빈 스택에 원소 A,B,C를 차례로 삽입(push) 후 한 번 삭제(pop)하는 연산 과정



<그림1_Python_알고리즘 기본_Stack 1_스택의 삽입 및 삭제 과정>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Stack 구현 실습

Push 연산

- append 메서드를 통해 리스트의 마지막에 데이터를 삽입

```
s = []
def my_push(item) :
    s.append(item)
```

- 인덱스 연산을 활용한 구현

```
def my_push(item, size):
    global top
    top += 1
    if top==size:
        print('Overflow!')
    else:
        stack[top] = item
```

삽입 :
저장소에 자료를 저장하는 연산

<개념5_알고리즘 기본_Stack 1_삽입>

단순한 Push 연산

- 크기가 정해진 리스트와 인덱스 연산을 활용

```
size = 10
stack = [0] * size
top = -1

my_push(10, size)       # my_push 함수를 이용
top += 1          # push(20)
stack[top] = 20
```

Pop 연산

- 남은 데이터 중 가장 늦게 저장된 데이터를 삭제하는 연산

```
def my_pop() :
    if len(s) == 0 :
        print('Underflow!')
        return
    else :
        return s.pop()  # 리스트 s의 마지막 원소 삭제
```

삭제 :
저장소에서 자료를 꺼내는 연산

<개념6_알고리즘 기본_Stack 1_삭제>

인덱스 연산을 이용한 Pop 연산

- 크기가 정해진 리스트와 인덱스 활용

```
def my_pop() :
    global top
    if top == -1 :
        print('Underflow')
        return 0
    else :
        top -= 1
        return stack[top+1]

print(my_pop())
```

```
if top > -1:     # pop()
    top -= 1
    print(stack[top+1])
```

스택 구현 시 고려 사항

- 1차원 배열을 사용하여 구현할 경우
  - 장점: 구현이 용이합니다.
  - 단점: 스택의 크기를 변경하기가 어렵습니다.

- 해결 방법: 저장소를 동적으로 할당하여 스택을 구현하는 방법
  (동적 연결 리스트를 이용하여 구현하는 방법)
  - 장점: 메모리를 효율적으로 사용합니다.
  - 단점: 구현이 복잡합니다.

동적 메모리 할당 :
필요한 만큼 메모리를 할당하고 해제하여 유연하게 크기를 조절할 수 있음

<개념7_알고리즘 기본_Stack 1_동적 메모리 할당>

연결리스트 :
데이터와 다음 노드의 주소를 함께 저장하여 요소들이 순차적으로 연결된 자료구조

<개념8_알고리즘 기본_Stack 1_연결리스트>

메모리 :
컴퓨터에서 데이터를 저장하고 처리하기 위해 사용하는 일시적인 저장 공간

<개념9_알고리즘 기본_Stack 1_메모리>

연습문제1

- 스택을 구현한 후, 이를 이용해 3개의 데이터를 저장하고 다시 3번 꺼내어 출력해 봅시다.
- 코드 : 4_Stack1/stack.py

Stack의 응용

괄호 검사

괄호의 종류

- 대괄호 ('[', ']'), 중괄호 ('{', '}'), 소괄호 ('(', ')')

조건

① 왼쪽 괄호의 개수와 오른쪽 괄호의 개수가 같아야 한다.

② 같은 괄호에서 왼쪽 괄호는 오른쪽 괄호보다 먼저 나와야 한다.

③ 괄호 사이에는 포함 관계만 존재한다.

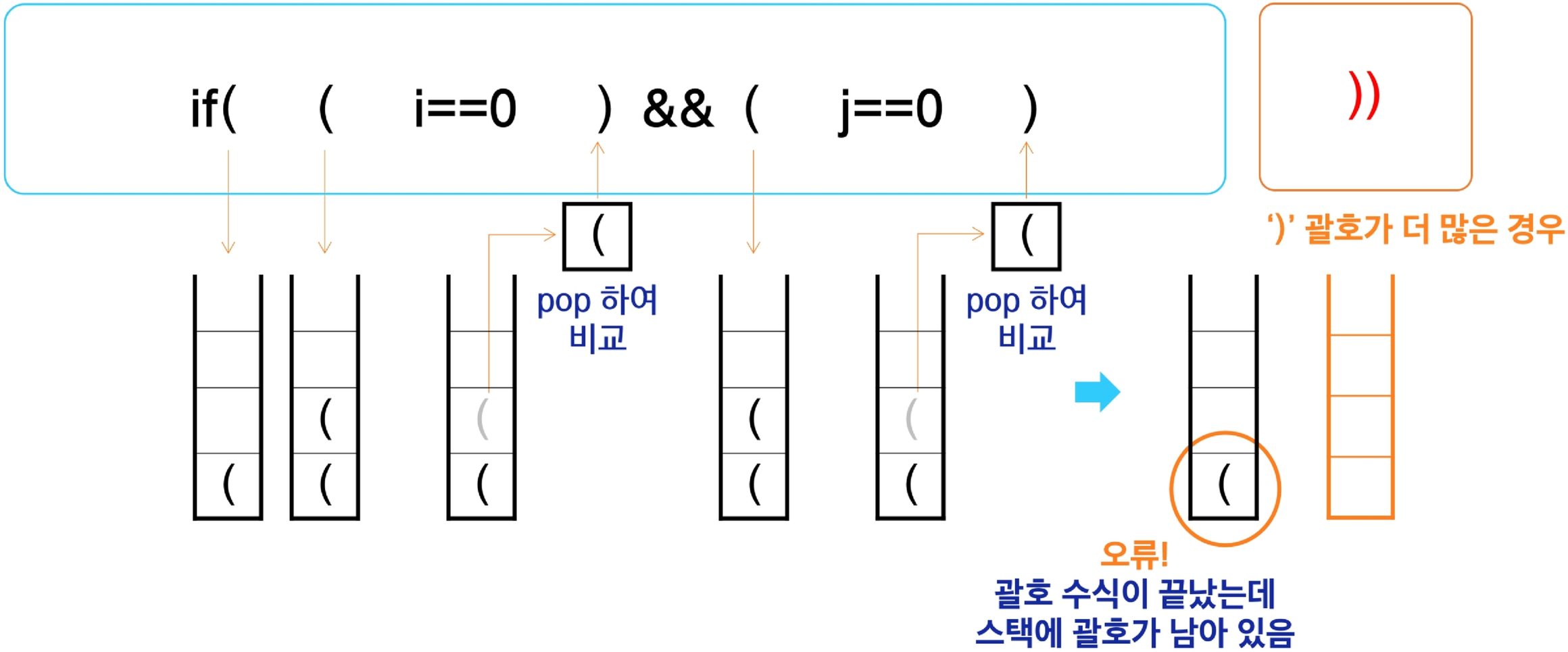
잘못된 괄호 사용의 예

(a(b)

a(b)c)

a{b(c[d]e}f)

스택을 이용한 괄호 검사



괄호를 검사하는 알고리즘 개요

1. 문자열에 있는 괄호를 차례대로 검사하면서 왼쪽 괄호를 만나면 스택에 삽입하고,
   오른쪽 괄호를 만나면 스택에서 top 괄호를 삭제한 후 오른쪽 괄호와 짝이 맞는지를 검사한다.

2. 이때, 스택이 비어 있으면 조건 ① 또는 조건 ②에 위배되고 괄호의 짝이 맞지 않으면 조건 ③에 위배된다.

3. 마지막 괄호까지 조사한 후에도 스택에 괄호가 남아 있으면 조건 ①에 위배된다.

연습문제2

괄호의 짝을 검사하는 프로그램을 작성하고, 이를 이용해 주어진 괄호 문자열의 올바른 사용 여부를 검사해 봅시다.
(코드 : 4_Stack1/ex2.py)

```
( )( )(( ))
(( )(( )( )(( )( ))(( ))))
```

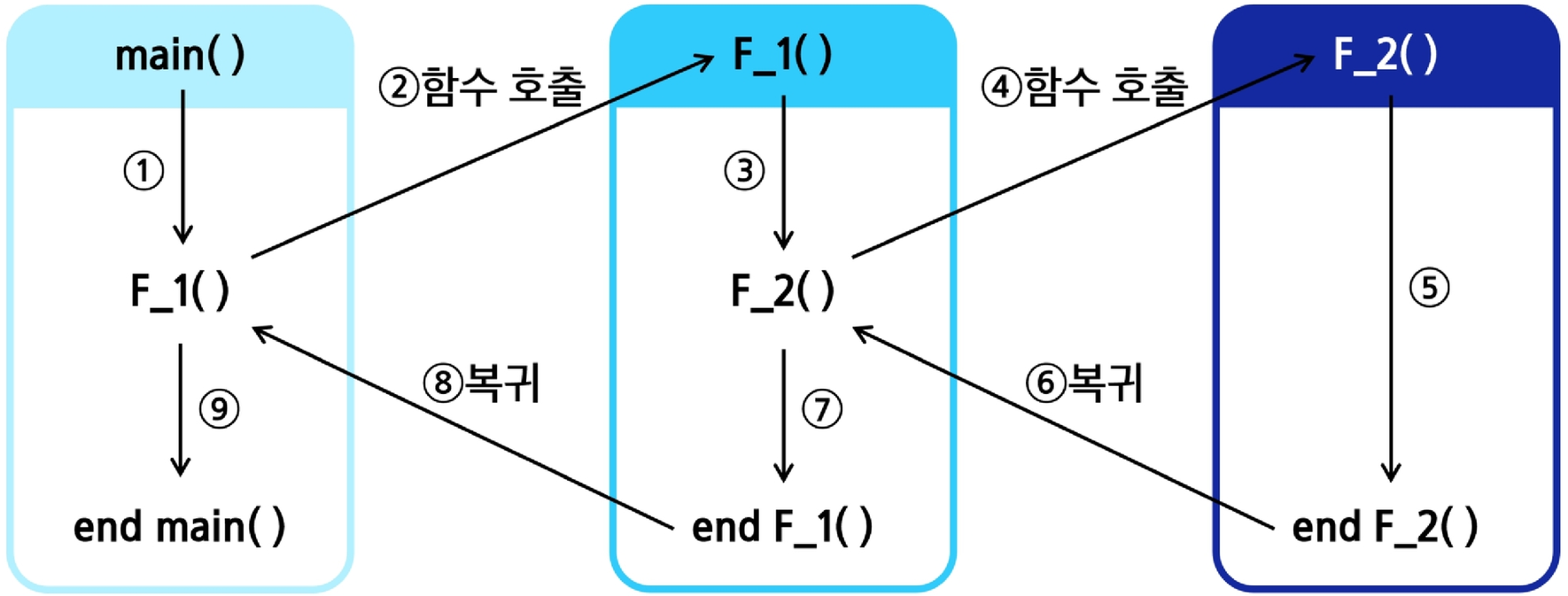
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Function Call

Function call

- 프로그램에서의 함수 호출과 복귀에 따른 수행 순서를 관리



<그림3_Python_알고리즘 기본_Stack 1_function call1>

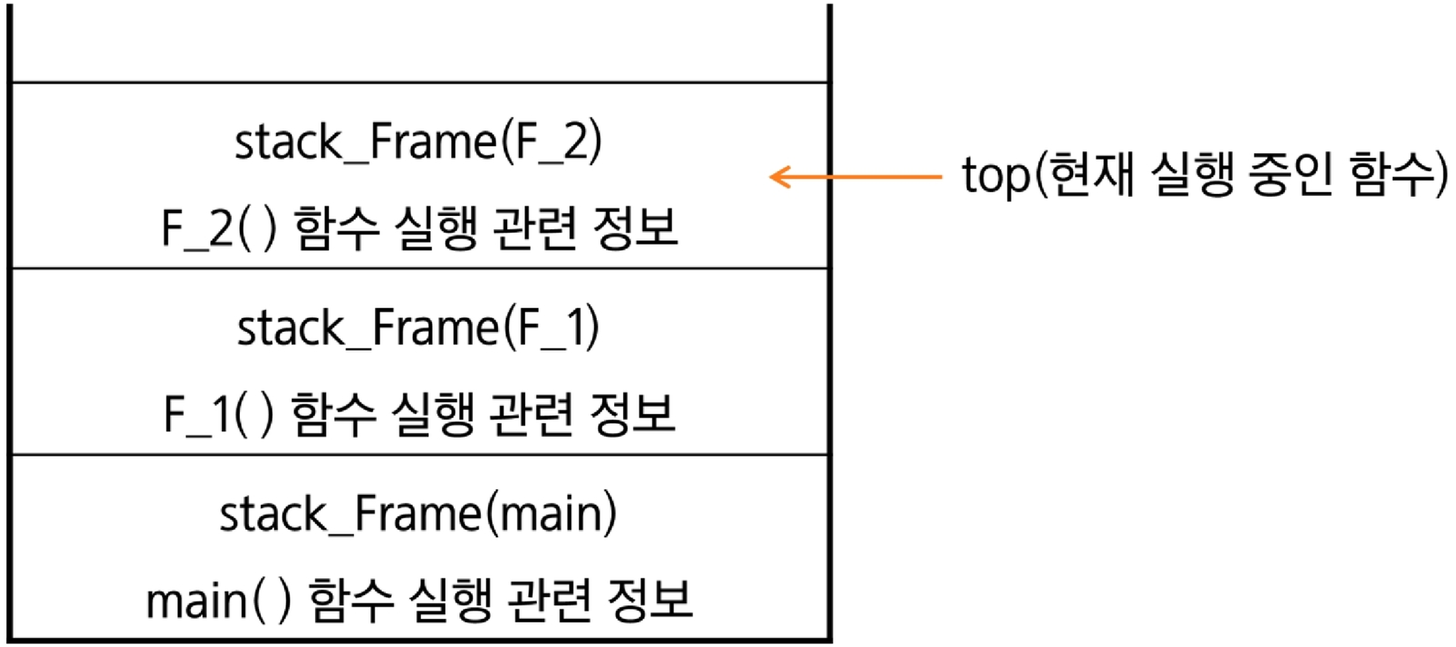
후입선출 :
가장 마지막에 넣은 자료가 가장 먼저 나오는 것

<개념10_알고리즘 기본_Stack 1_후입선출>

가장 마지막에 호출된 함수가 가장 먼저 실행을 완료하고 복귀하는 후입선출 구조이므로,
후입선출 구조의 스택을 이용하여 수행 순서 관리

Function call

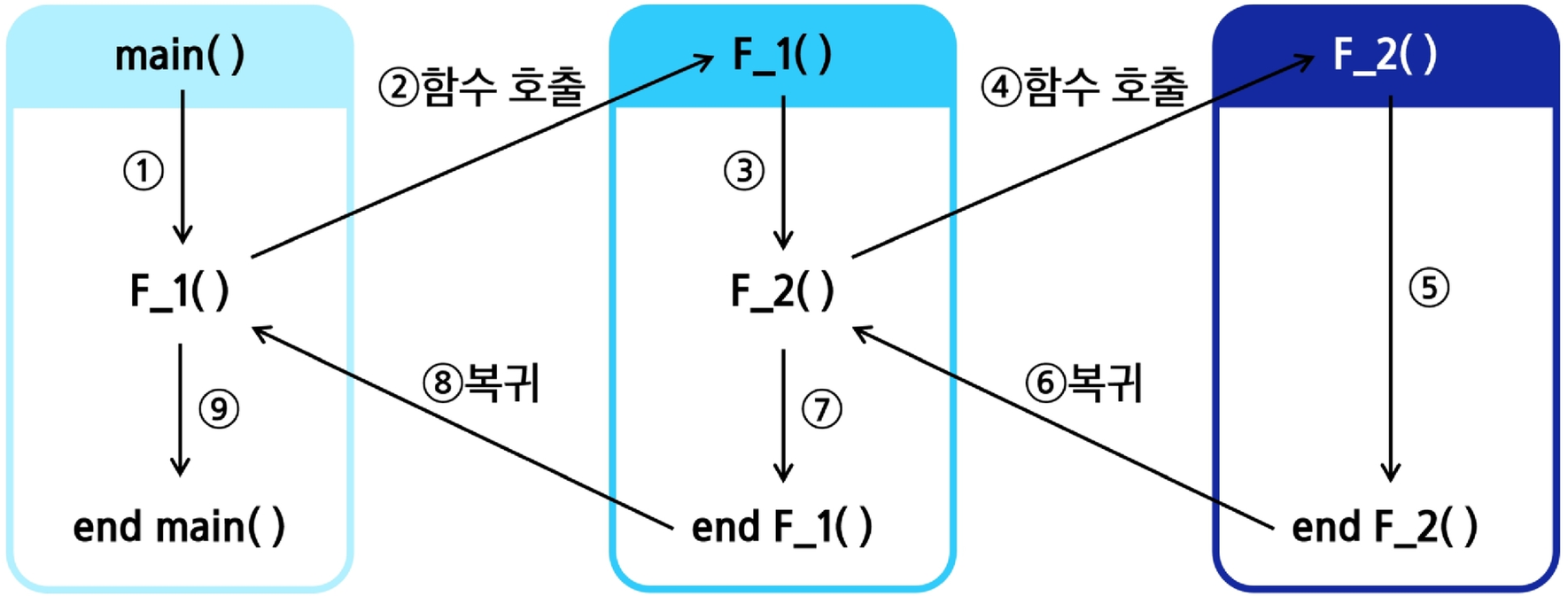
- 시스템 스택
  - 함수 수행에 필요한 지역 변수, 매개변수 및 수행 후 복귀할 주소 등의 정보를 저장합니다.
  - 함수 호출이 발생하면 스택 프레임(stack frame)에 저장하여 시스템 스택에 삽입합니다.



<그림4_Python_알고리즘 기본_Stack 1_시스템스택>

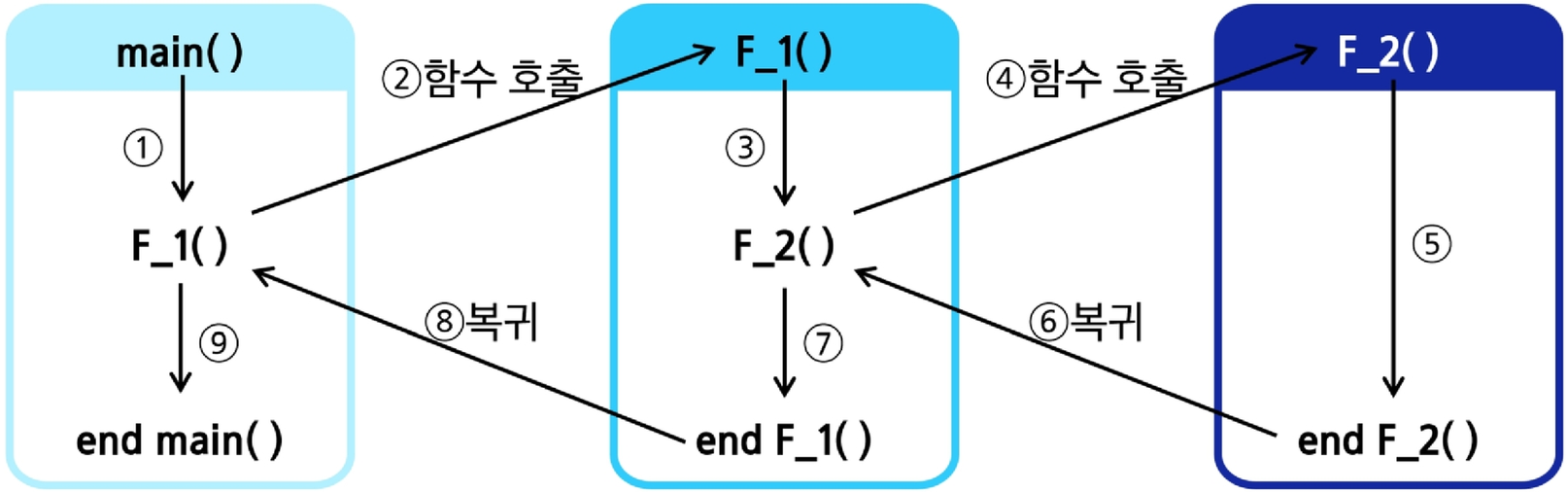
Function call

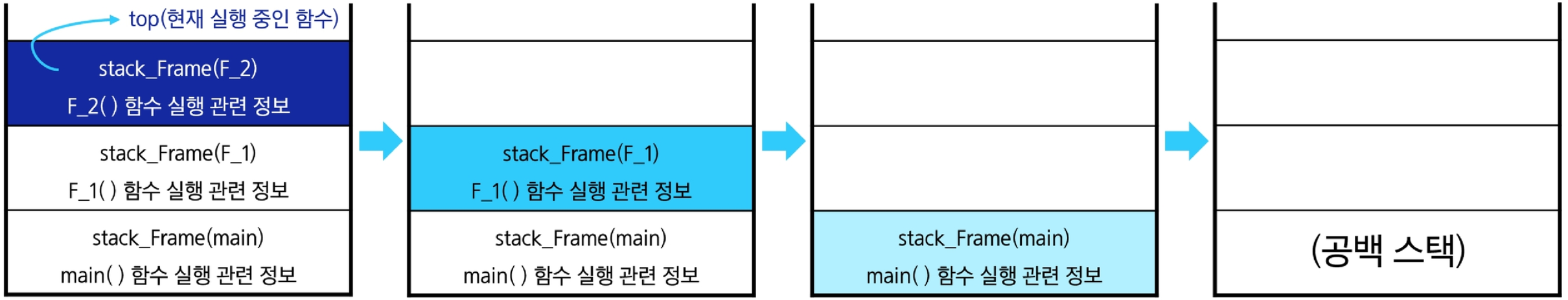
프로그램에서의 함수 호출과 복귀에 따른 수행 순서를 관리
- 함수의 실행이 끝나면 시스템 스택의 top 원소(스택 프레임)를 삭제(pop)하면서 프레임에 저장되어 있던 복귀 주소를 확인하고 복귀시킵니다.
- 함수 호출과 복귀에 따라 이 과정을 반복하여 전체 프로그램 수행이 종료되면 시스템 스택은 공백 스택이 됩니다.



<그림5_알고리즘 기본_Stack1_function call>

시스템 스택을 이용한 함수 호출과 복귀





<그림6_알고리즘 기본_Stack1_시스템 스택>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Stack 기반 문제 해결 기법

재귀호출

재귀호출의 개념과 활용

재귀호출

함수가 자신과 같은 작업을 반복해야 할 때, 자신을 다시 호출하는 구조

반복적으로 자기 자신을 호출해야 하거나 전체를 부분 문제로 나눌 수 있는 경우에는
일반적인 호출 방식보다 재귀호출 방식을 사용하여 함수를 만들면
프로그램의 크기를 줄이고 간단하게 작성할 수 있습니다. 예) factorial

n에 대한 factorial

- 1부터 n까지의 모든 자연수를 곱하여 구하는 연산

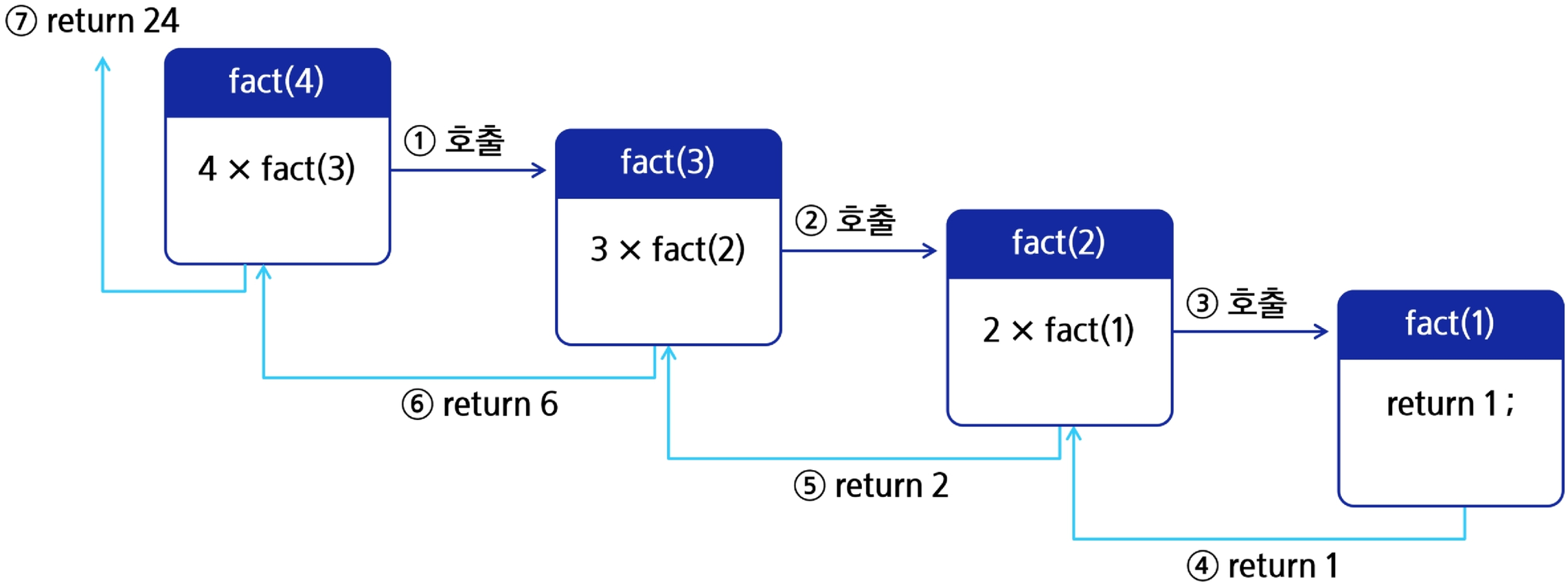
- 마지막에 구한 하위 값을 이용하여 상위 값을 구하는 작업을 반복합니다.

```
n! = n x (n-1)!
  (n-1)! = (n-1) x (n-2)!
  (n-2)! = (n-2) x (n-3)!


  …


  2! = 2 x 1!
  1! = 1
```

factorial 함수에서 n=4 인 경우의 실행



<그림7_알고리즘 기본_Stack1_factorial 함수 실행 과정>

피보나치 수열

피보나치 수열

0과 1로 시작하고 이전 두 수의 합을 다음 항으로 하는 수열

0, 1, 1, 2, 3, 5, 8, 13, ...

피보나치 수열의 수학적 정의

- 피보나치 수열의 i번 값을 계산하는 함수 F를 정의

  $F_0 = 0, F_1 = 1$

  $F_i = F_{i-1} + F_{i-2}$, for i ≥2

피보나치 수를 구하는 재귀함수

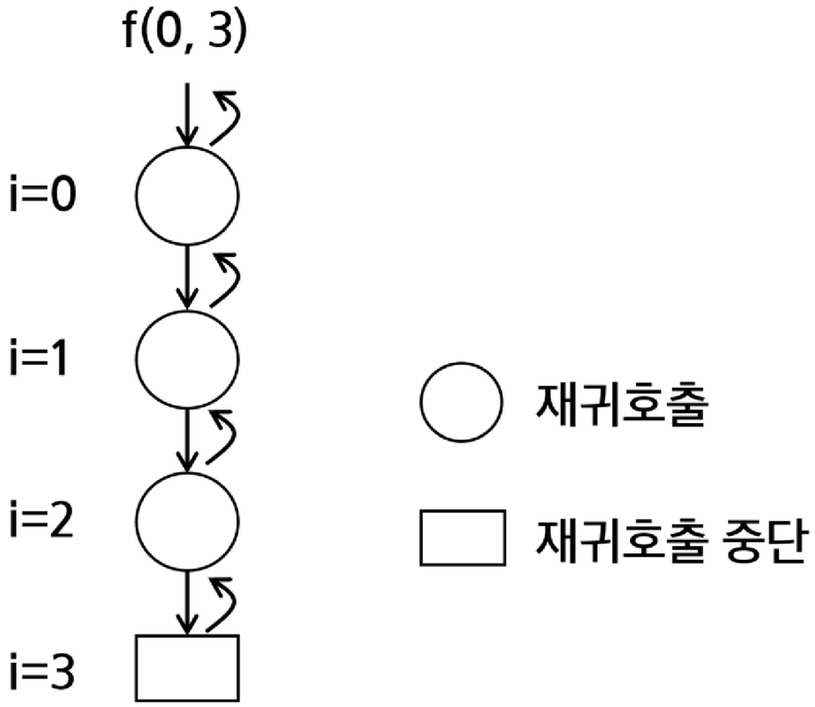
피보나치 수열의 수학적 정의에 따라 피보나치 수열의 i번 항을 반환하는 함수를 재귀 함수로 구현 가능

```
def fibo(n) :
    if n < 2 :
        return n
    else :
        return fibo(n-1) + fibo(n-2)
```

재귀함수의 기본형

- 현재 호출 단계와 목표 단계를 인자로 사용

- 현재 단계 i, 목표 N을 인자로 갖는 재귀 함수 f(i, N)이 기본형입니다. (코드 : 4_Stack1/recur1.py)



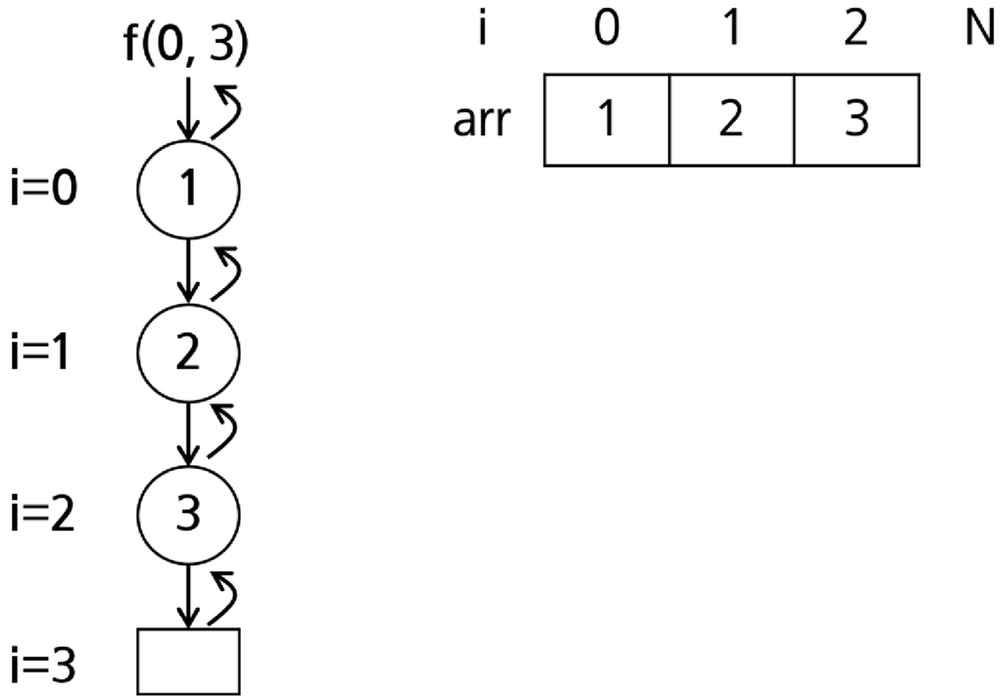

```
def f(i, N):
    if i==N:    # 중단조건
        return
    else:       # 재귀호출
        f(i+1, N)
```

<그림8_알고리즘 기본_Stack1_재귀함수의 기본형>

모든 배열 원소에 접근하는 재귀 함수

- 기본형 활용

 - 호출 단계 i 에서 arr[i] 원소에 접근하고, i가 배열의 크기와 같아지면 재귀호출을 중단합니다.

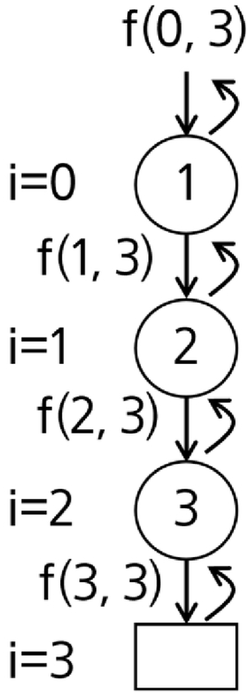


<그림9_알고리즘 기본_Stack1_배열 원소에 접근하는 재귀함수>

모든 배열 원소에 접근하는 재귀 함수

- 구현

i	0	1	2	N
arr	1	2	3	



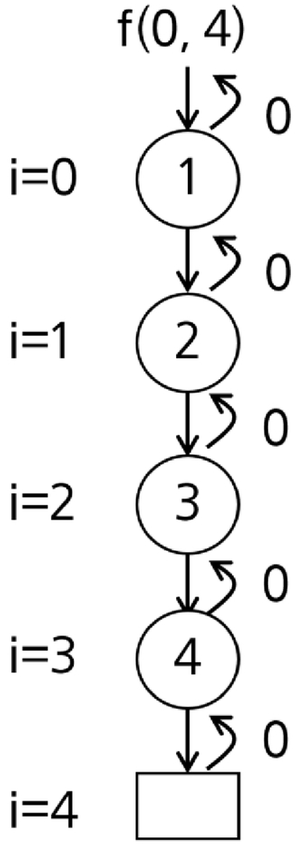

```
def f(i, N) : # 크기 N인 배열 arr[i]에 접근
    if i == N :
        return
    else :
        print(arr[i])
        f(i+1, N)
```

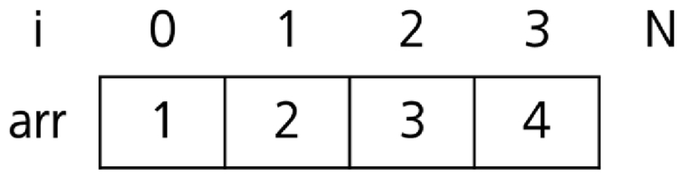
<그림10_알고리즘 기본_Stack1_모든 배열 원소에 접근하는 단계>

배열 원소 검색

- 배열에 v가 있으면 1, 없으면 0을 리턴하는 함수

- v = 5인 경우, arr에는 5가 없으므로 마지막 단계까지 호출하고 0을 리턴합니다.

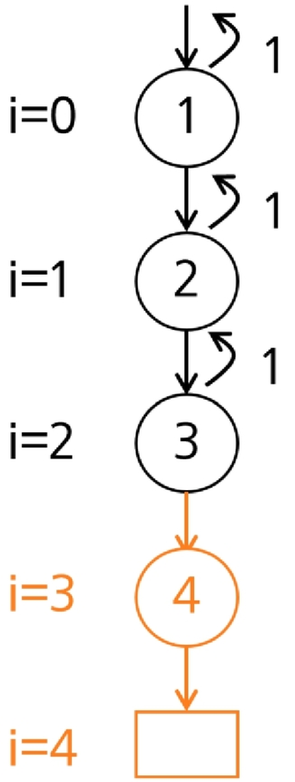




<그림11_알고리즘 기본_Stack1_배열 원소 검색>

배열 원소 v 검색

- v = 3인 경우
  - 3을 찾은 경우 재귀호출을 중단합니다.



i	0	1	2	3	N
arr	1	2	3	4	

```python
def f(i, N, v) :  # v 찾는 값
    if i == N :
        return 0
    elif arr[i] == v:
        return 1
    else :
        return f(i+1, N, v)
```

<그림12_Python_알고리즘 기본_Stack 1_배열에 v가 있는 경우>

SAMSUNG SW AI ACADEMY FOR YOUTH

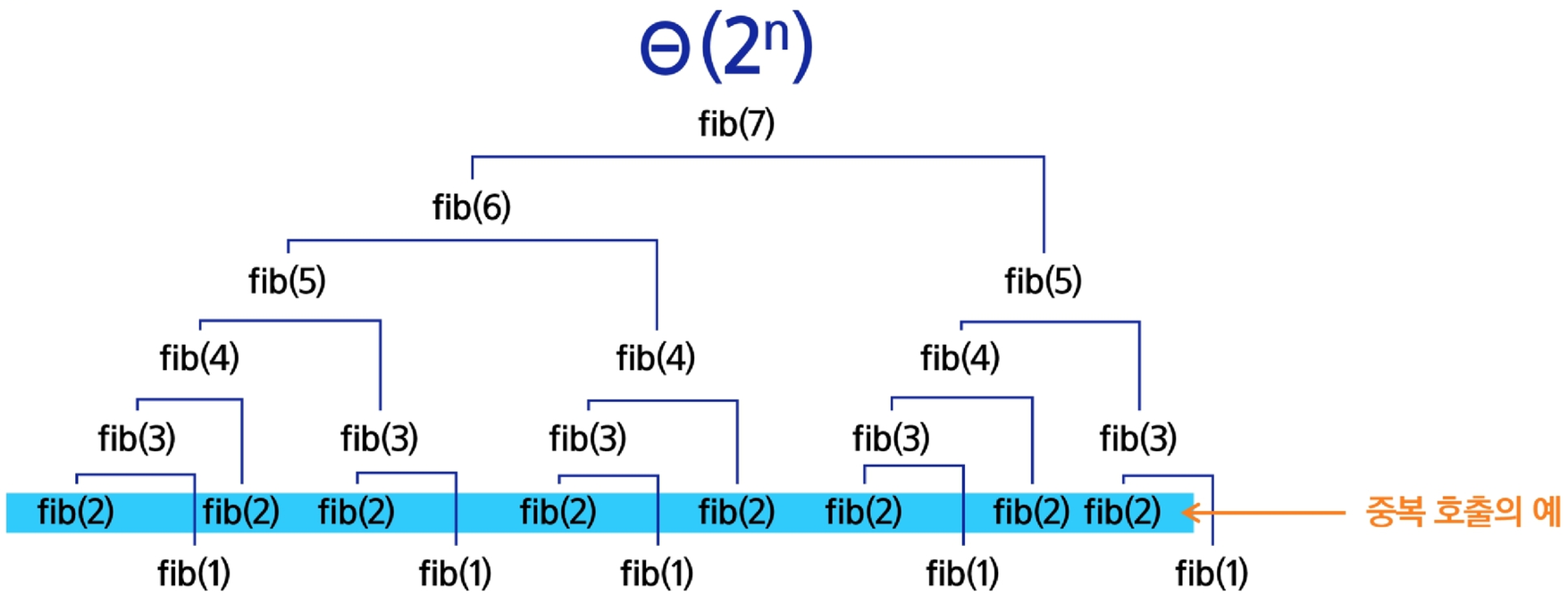
함께가요 미래로!
Enabling People

Memoization

피보나치 재귀호출의 문제점

피보나치 수를 구하는 함수를 재귀 함수로 구현한 알고리즘은 “엄청난 중복 호출이 존재한다”는 문제 발생

피보나치 수열의 Call Tree



<그림13_Python_알고리즘 기본_Stack 1_피보나치 수열의 Call Tree>

재귀 알고리즘 최적화: 메모이제이션 기법

메모이제이션
Memoization

컴퓨터 프로그램을 실행할 때 이전에 계산한 값을 메모리에 저장해서
매번 다시 계산하지 않도록 하여 전체적인 실행 속도를 빠르게 하는 기술

동적 계획법의 핵심이 되는 기술로, 앞의 예시 피보나치 수를 구하는 알고리즘에서
fibo(n)의 값을 계산하자마자 저장하면(memoize), 실행 시간을 Θ(n)으로 줄일 수 있습니다.

Memoization을 적용한 피보나치

계산된 적이 있는 피보나치 수는 저장된 값을 리턴

```
#memo를 위한 배열을 할당하고, 모두 0으로 초기화 한다;
#memo[0]을 0으로 memo[1]는 1로 초기화한다;

def fibo1(n) :
    if n >= 2 and memo[n] == 0 :
        memo[n] = fibo1(n-1) + fibo1(n-2)
    return memo[n]

memo = [0] * (n+1)
memo[0] = 0
memo[1] = 1
```

DP

최적화를 위한 알고리즘: 동적계획법(DP)

동적 계획법

DP, Dynamic Programming

입력 크기가 작은 부분 문제들을 먼저 해결한 뒤, 그 결과를 바탕으로 더 큰 부분 문제를
차례대로 해결해 나가며 최종적으로 전체 문제의 해답을 도출하는 알고리즘

동적 계획 (Dynamic Programming) 알고리즘은 그리디 알고리즘과 같이 최적화 문제를 해결하는 알고리즘이다.

피보나치 수열

DP 적용 문제

- 문제의 최적 해가 그 하위 문제의 최적 해로부터 쉽게 구성될 수 있는 최적 부분 구조여야 합니다.
- 동일한 하위 문제가 여러 번 반복되어 나타나는 중복 부분 문제여야 합니다.

최적 부분 구조 + 중복 부분 문제

F (n) = F(n-1) + F(n-2)

= (F(n-2) + F(n-3)) + F(n-2)

= …

F(5) = F(4) + F(3)

= (F(3) + F(2)) + (F(2) + F(1))

= ((F(2) + F(1)) + (F(1) + F(0))) + ((F(1) + F(0)) + 1)

피보나치 수열 함수의 중복 호출 제거

부분 문제로 나누는 일을 끝냈으면 가장 작은 부분 문제부터 해를 구함.

그 결과는 테이블에 저장하고, 테이블에 저장된 부분 문제의 해를 이용하여 상위 문제의 해를 구함.

테이블 인덱스	저장되어 있는 값
[0]	0
[1]	1
[2]	1
[3]	2
[4]	3
…	…
[n]	fibo(n)

<표1_Python_ 알고리즘 기본_피보나치 수 DP 적용>

피보나치 수열을 DP로 구현한 코드

```
def fibo2(n) :
    f = [0] * (n + 1)
    f[0] = 0
    f[1] = 1
    for i in range(2, n + 1) :
        f[i] = f[i-1] + f[i-2]

    return f[n]
```

DP의 구현 방식

Recursive 방식 : fibo1()

Iterative 방식 : fibo2()

오버헤드 :
추가적인 자원 소모나 처리 시간을 발생시키는 부가적인 비용

<개념11_알고리즘 기본_Stack 1_오버헤드>

재귀 구조에 Memoization을 사용하는 것보다, 반복적인 구조로 DP를 구현하는 방식이 성능 면에서 더 효율적

재귀적 구조는 내부에 시스템 호출 스택을 사용하는 오버헤드가 발생하기 때문

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

DFS

비선형 자료구조 탐색

깊이 우선 탐색

DFS, Depth First Search

한 방향으로 가능한 한 깊게 탐색한 후, 더 이상 갈 곳이 없으면 되돌아와 다른 방향을 탐색

비선형 구조인 그래프 구조는 그래프로 표현된 모든 자료를 빠짐없이 검색하는 것이 중요합니다.
이러한 탐색에는 다음의 두 가지 방법이 사용됩니다.
① 깊이 우선 탐색(Depth First Search, DFS) ② 너비 우선 탐색(Breadth First Search, BFS)

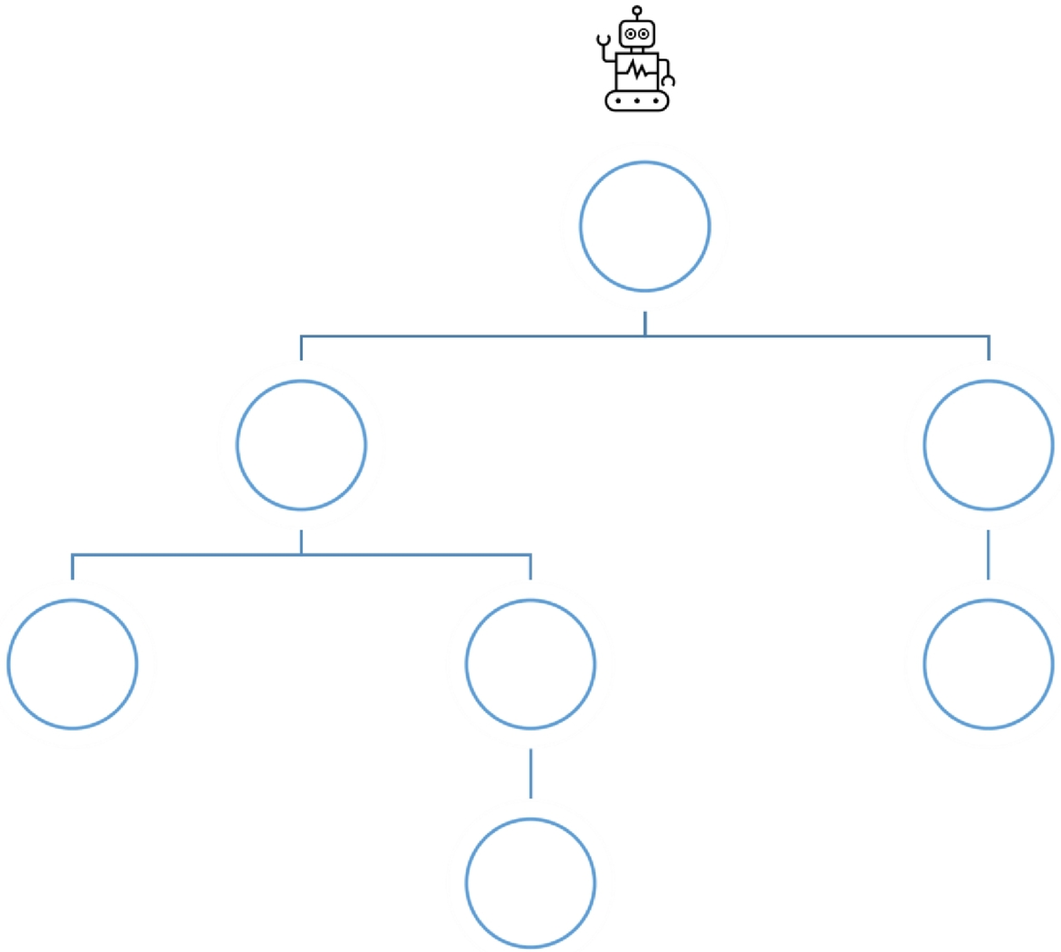
DFS의 동작 원리

① 시작 정점의 한 방향으로 갈 수 있는 경로가 있는 곳까지 깊이 탐색해 나간다.

② 더 이상 갈 곳이 없게 되면, 가장 마지막에 만났던 갈림길 간선이 있는 정점으로 되돌아와서 다른 방향의 정점으로 탐색을 계속 반복하여 결국 모든 정점을 방문하는 탐색 방법

가장 마지막에 만났던 갈림길의 정점으로 되돌아가서
다시 깊이 우선 탐색을 반복해야 하므로 후입선출 (LIFO) 구조의 스택 사용

로봇이 선을 따라 모든 칸을 탐색하는 방법



<그림14_Python_알고리즘 기본_Stack 1_로봇이 선을 따라 모든 칸을 탐색하는 방법1>

DFS 알고리즘

① 시작 정점 v를 결정하여 방문한다.

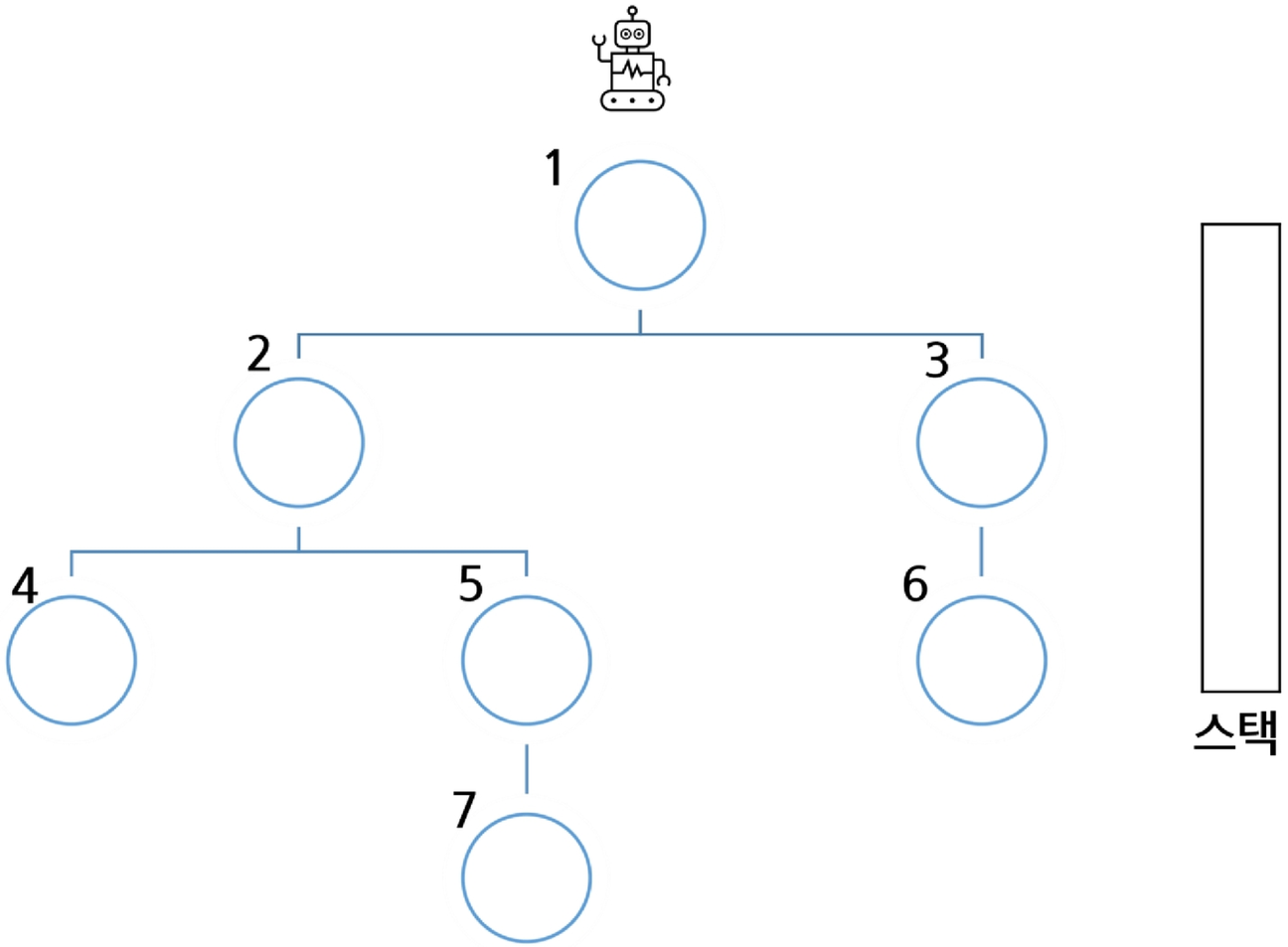
② 정점 v에 인접한 정점 중에서

a. 방문하지 않은 정점 w가 있으면,
정점 v를 스택에 push하고 정점 w를 방문한다.
그리고 w를 v로 하여 다시 ②를 반복한다.

b. 방문하지 않은 정점이 없으면, 탐색의 방향을
바꾸기 위해서 스택을 pop하여 받은 가장 마지막
방문 정점을 v로 하여 다시 ②를 반복한다.

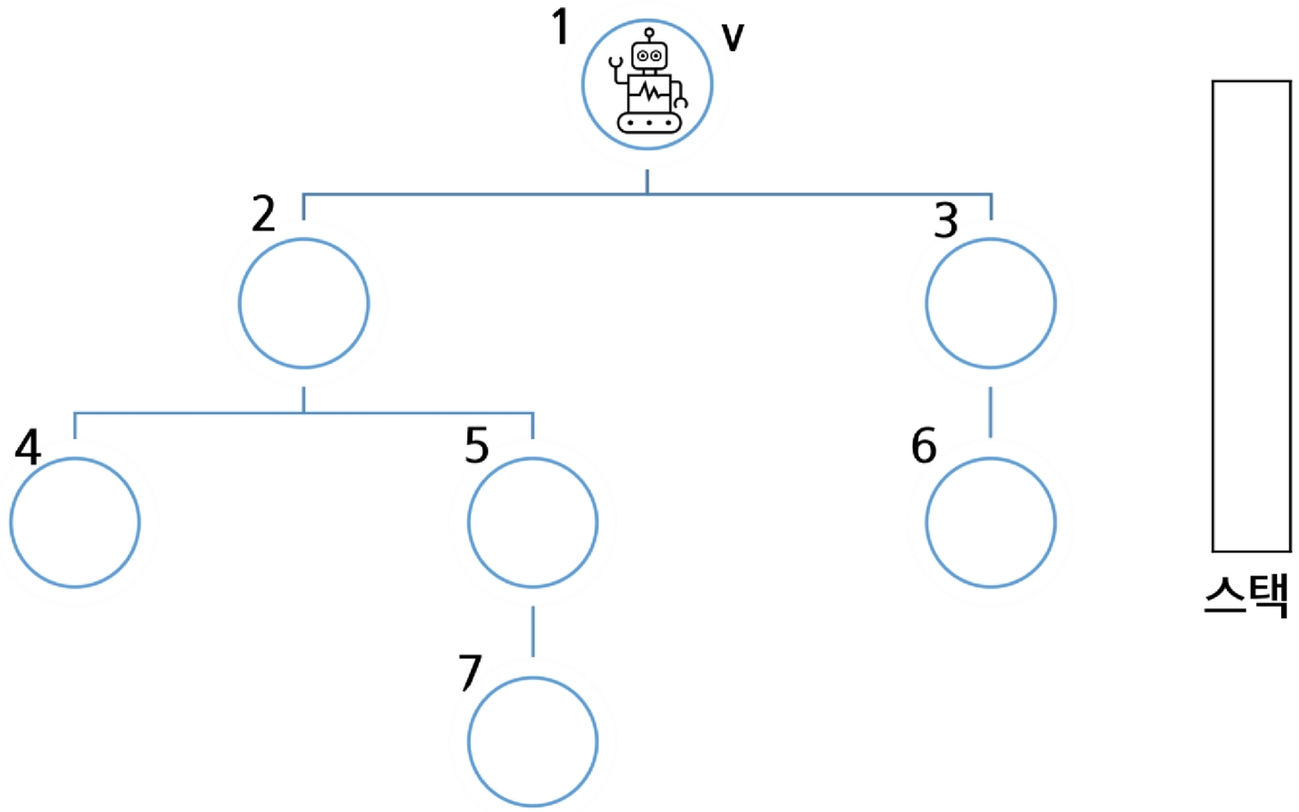
③ 스택이 공백이 될 때까지 ②를 반복한다.

로봇이 선을 따라 모든 칸을 탐색하는 방법



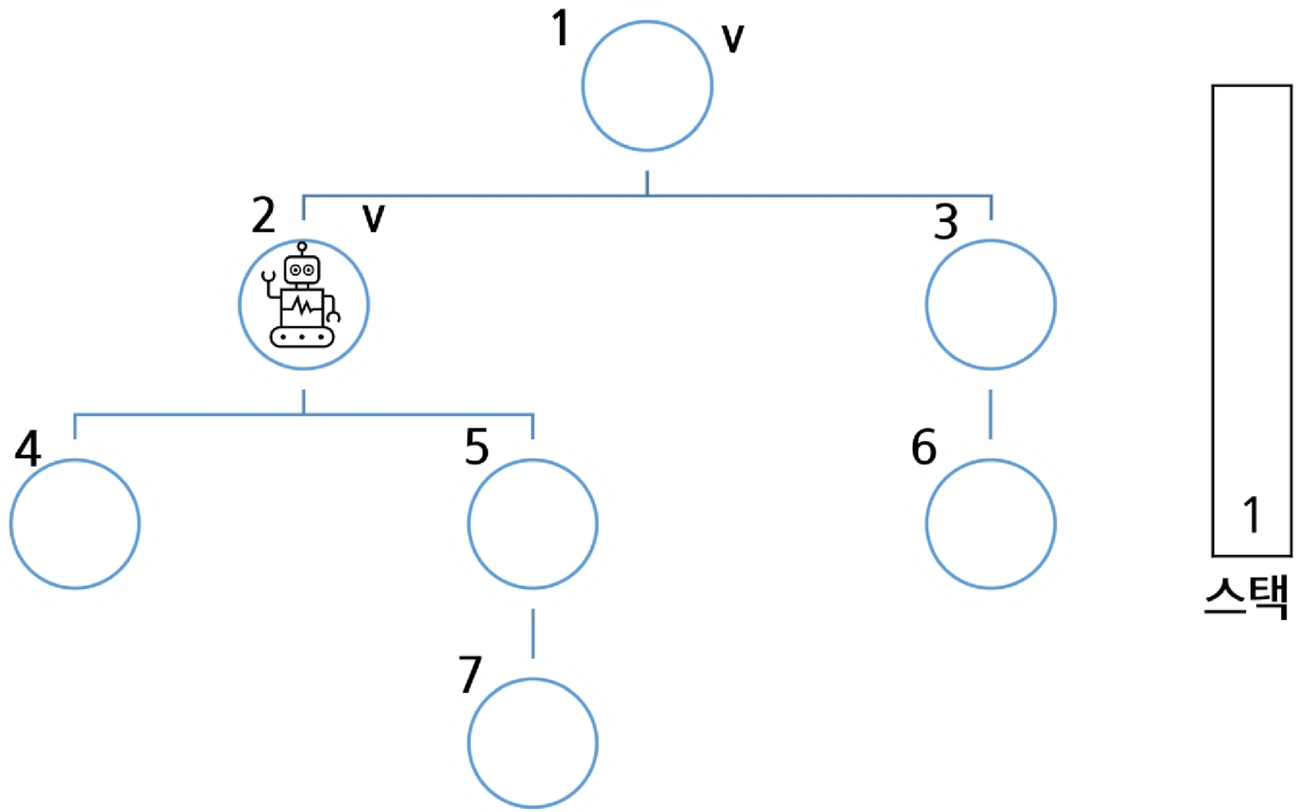
<그림15_Python_알고리즘 기본_Stack 1_로봇이 선을 따라 모든 칸을 탐색하는 방법2>

로봇이 선을 따라 모든 칸을 탐색하는 방법



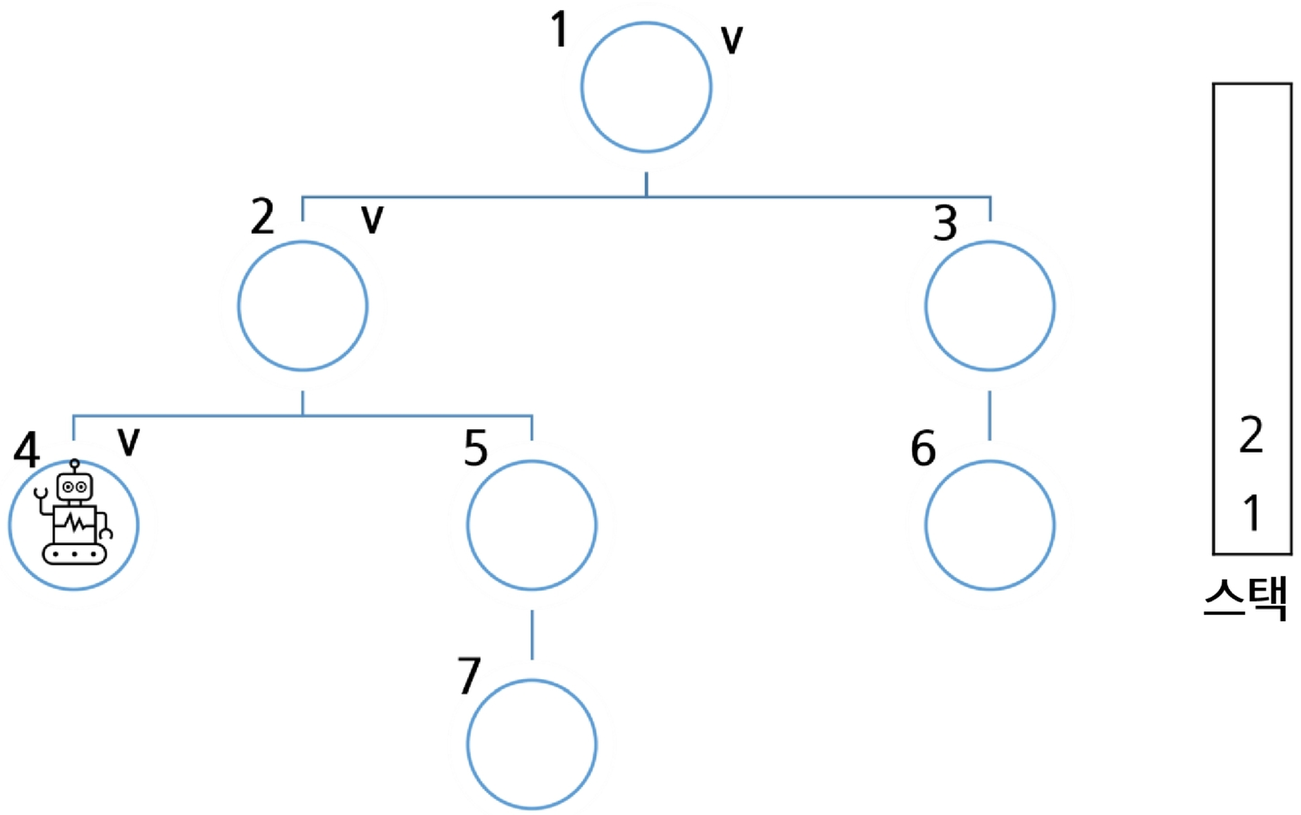
<그림16_Python_알고리즘 기본_Stack 1_로봇이 선을 따라 모든 칸을 탐색하는 방법3>

로봇이 선을 따라 모든 칸을 탐색하는 방법



<그림17_Python_알고리즘 기본_Stack 1_로봇이 선을 따라 모든 칸을 탐색하는 방법4>

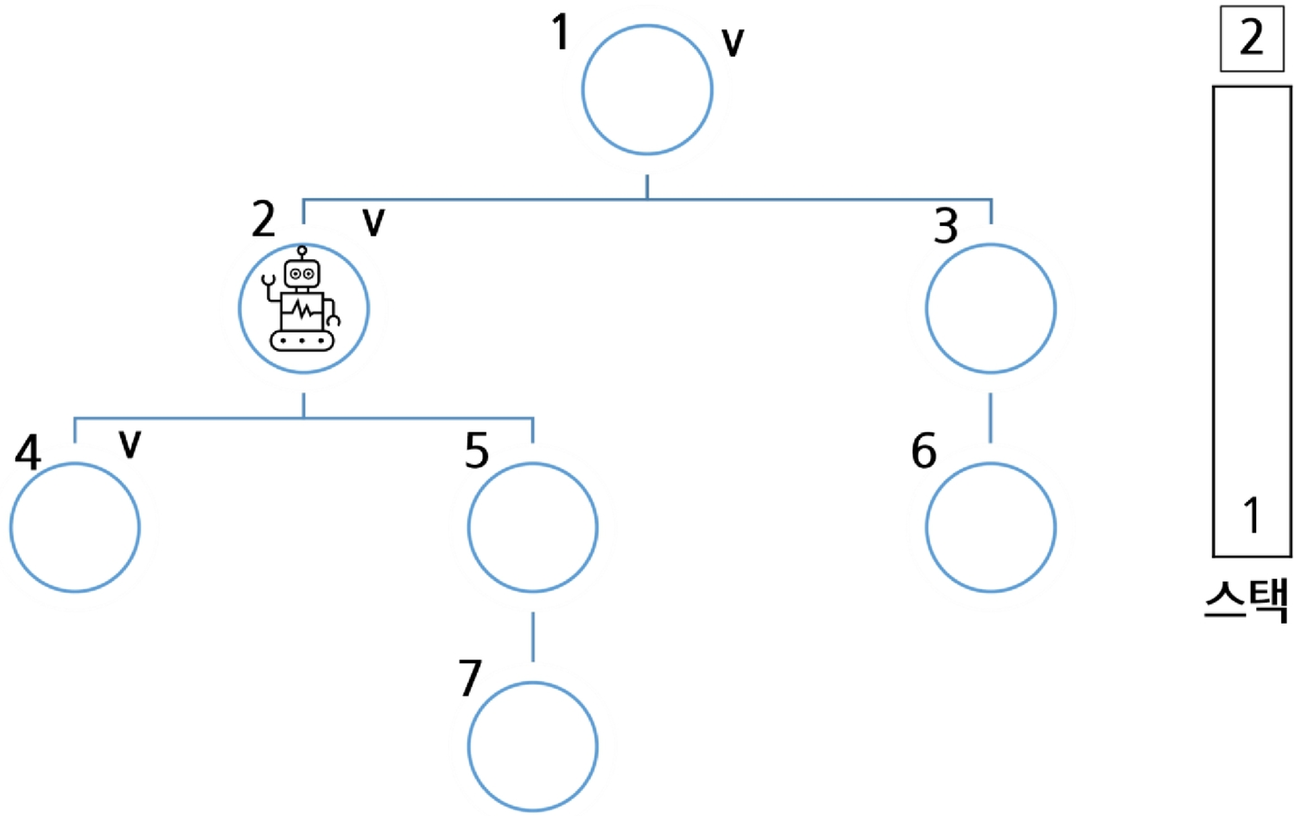
로봇이 선을 따라 모든 칸을 탐색하는 방법



<그림18_Python_알고리즘 기본_Stack 1_로봇이 선을 따라 모든 칸을 탐색하는 방법5>

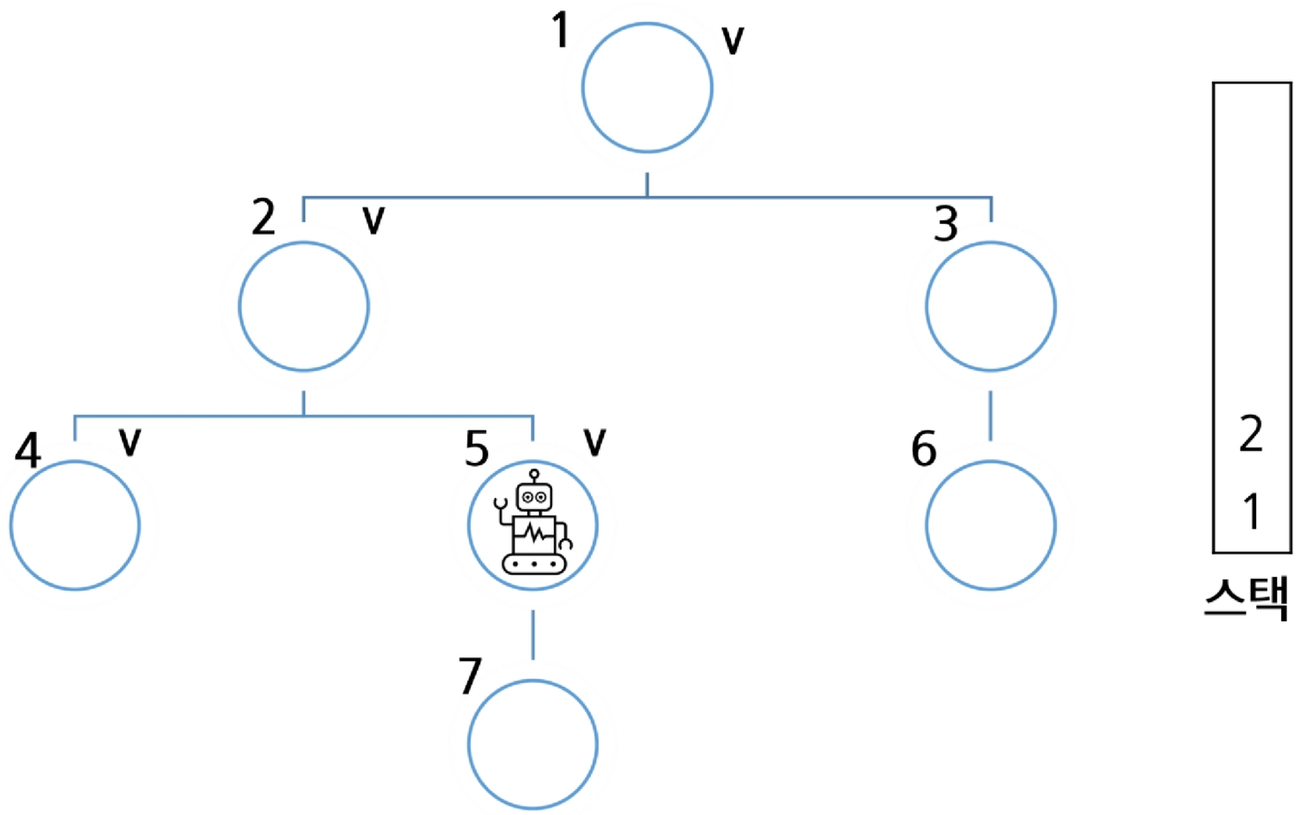
이론

로봇이 선을 따라 모든 칸을 탐색하는 방법



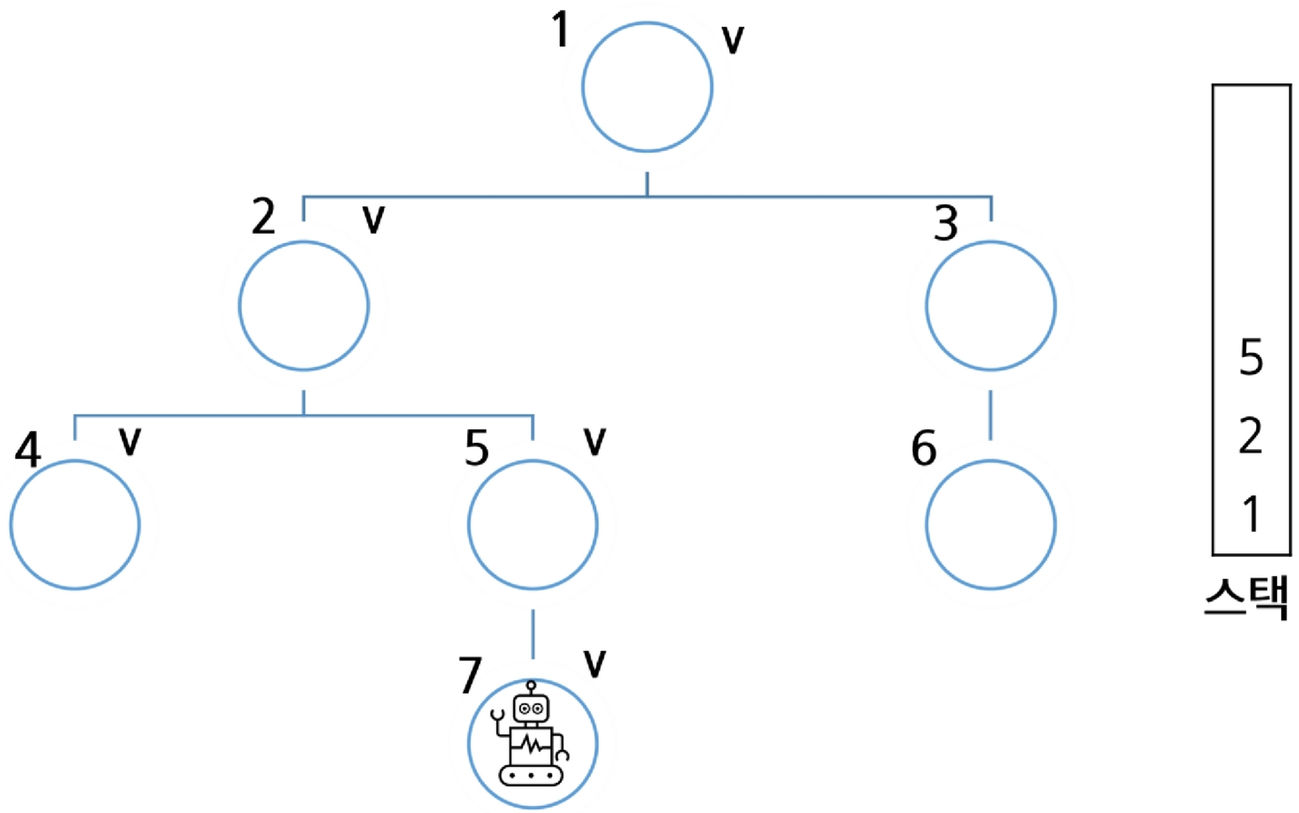
<그림19_Python_알고리즘 기본_Stack 1_로봇이 선을 따라 모든 칸을 탐색하는 방법6>

로봇이 선을 따라 모든 칸을 탐색하는 방법



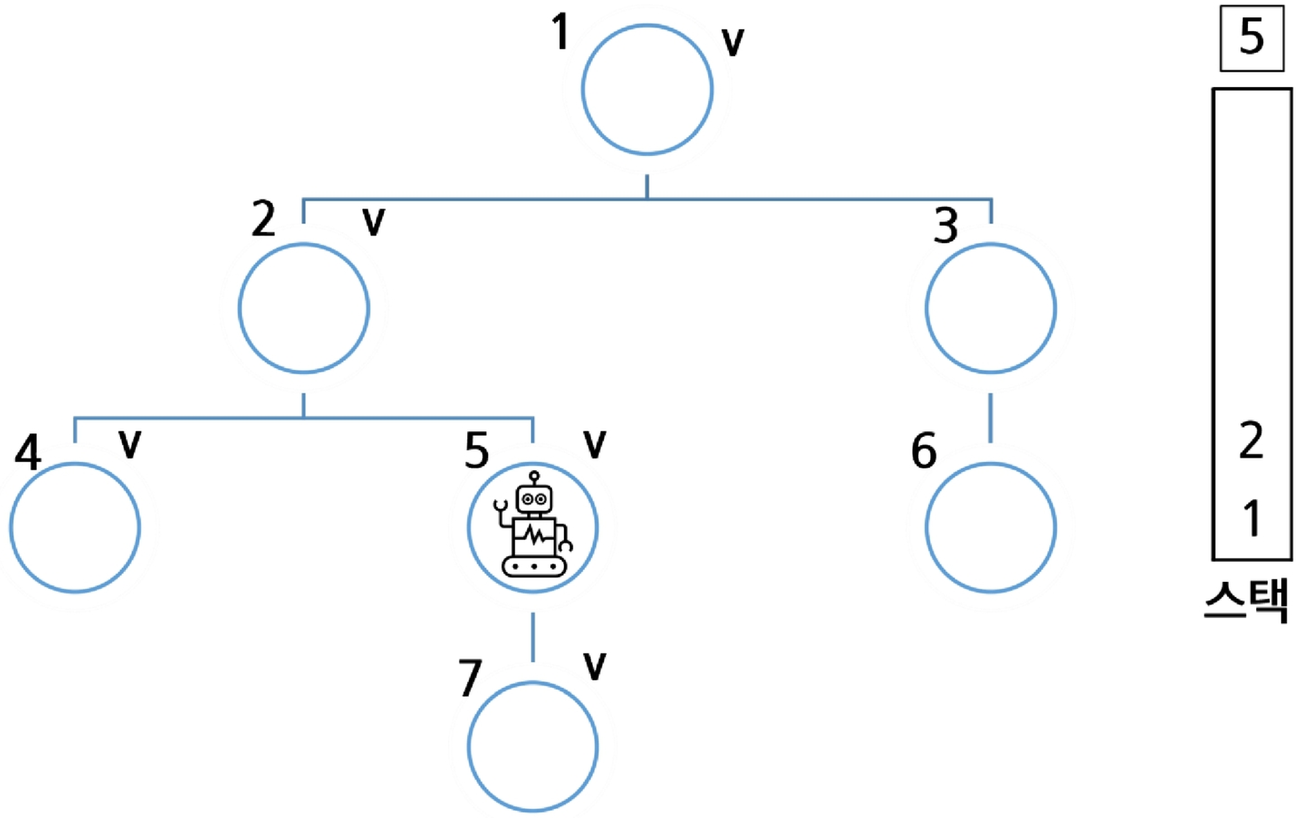
<그림20_Python_알고리즘 기본_Stack 1_로봇이 선을 따라 모든 칸을 탐색하는 방법7>

로봇이 선을 따라 모든 칸을 탐색하는 방법



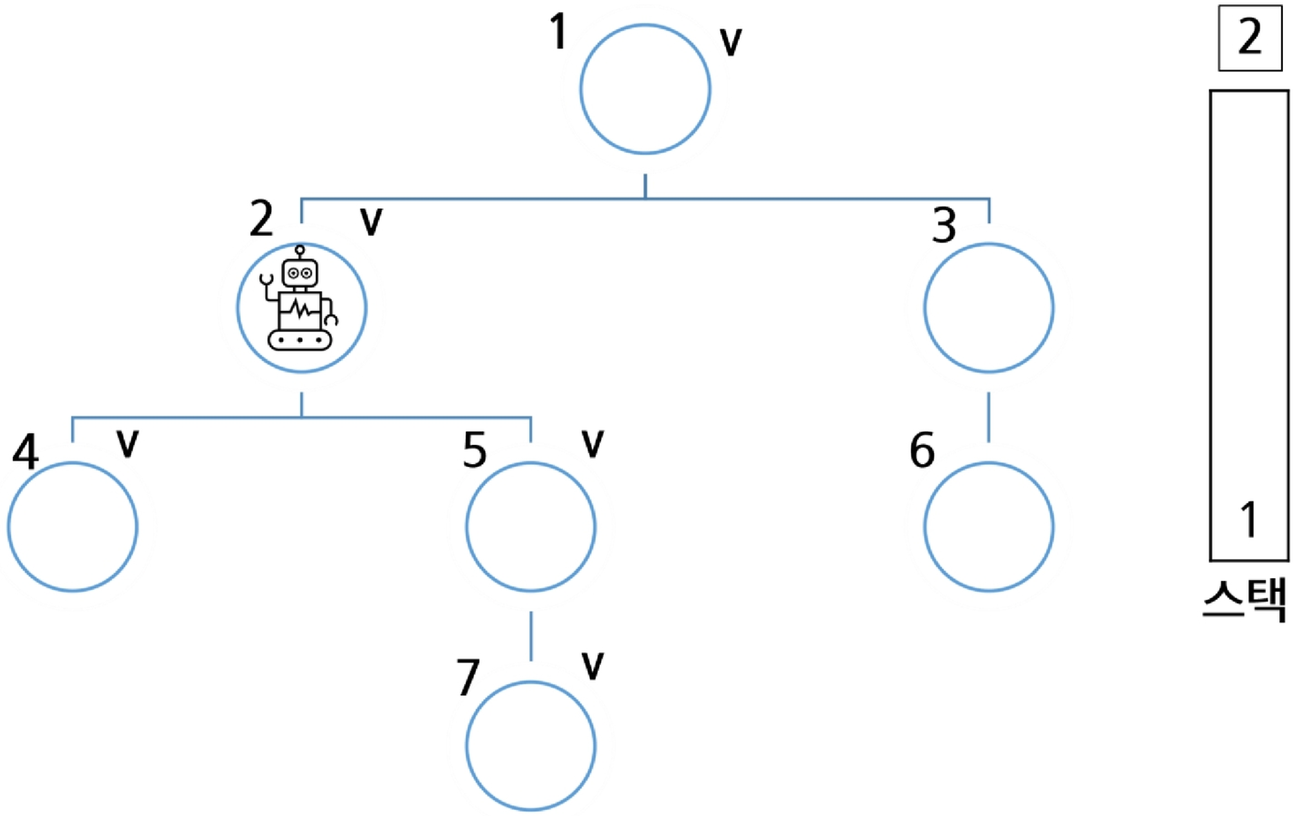
<그림21_Python_알고리즘 기본_Stack 1_로봇이 선을 따라 모든 칸을 탐색하는 방법8>

로봇이 선을 따라 모든 칸을 탐색하는 방법



<그림22_Python_알고리즘 기본_Stack 1_로봇이 선을 따라 모든 칸을 탐색하는 방법9>

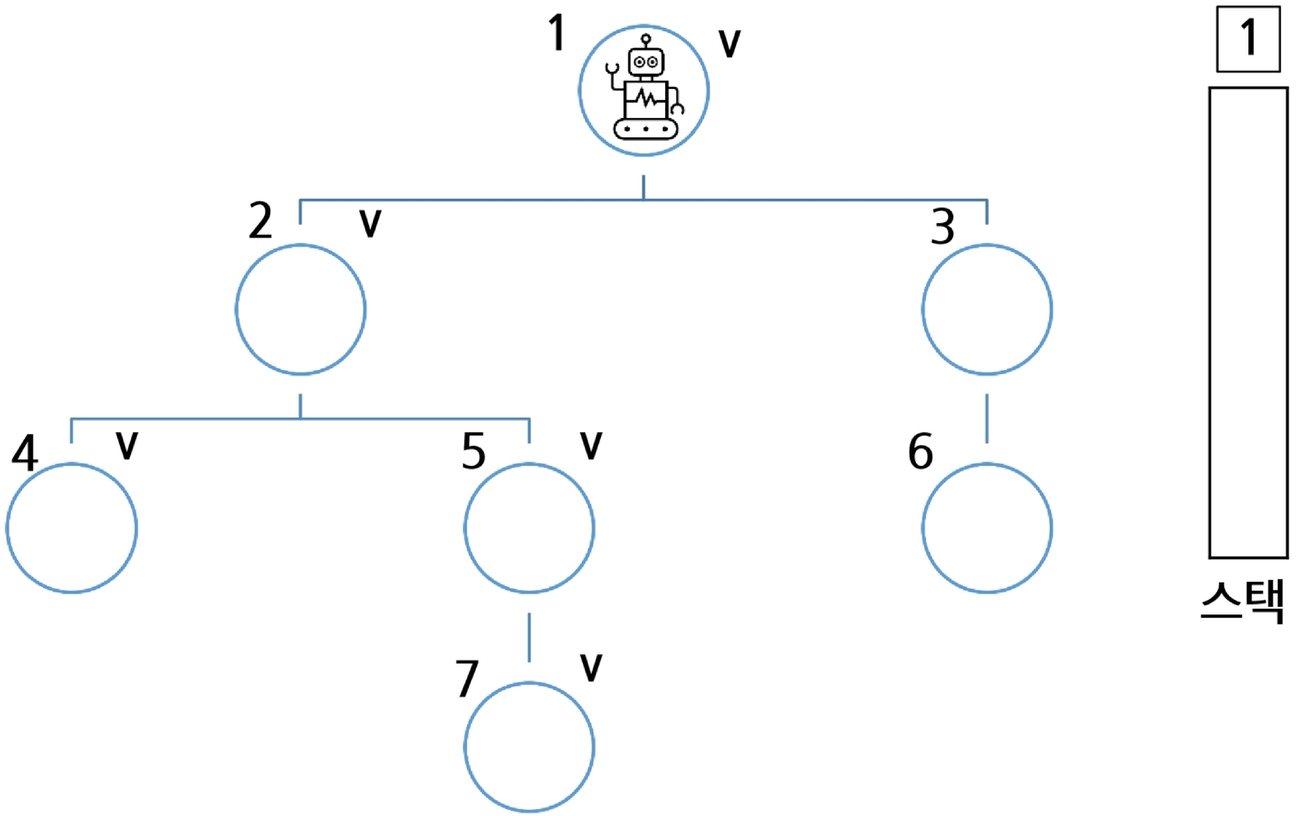
로봇이 선을 따라 모든 칸을 탐색하는 방법



<그림23_Python_알고리즘 기본_Stack 1_로봇이 선을 따라 모든 칸을 탐색하는 방법10>

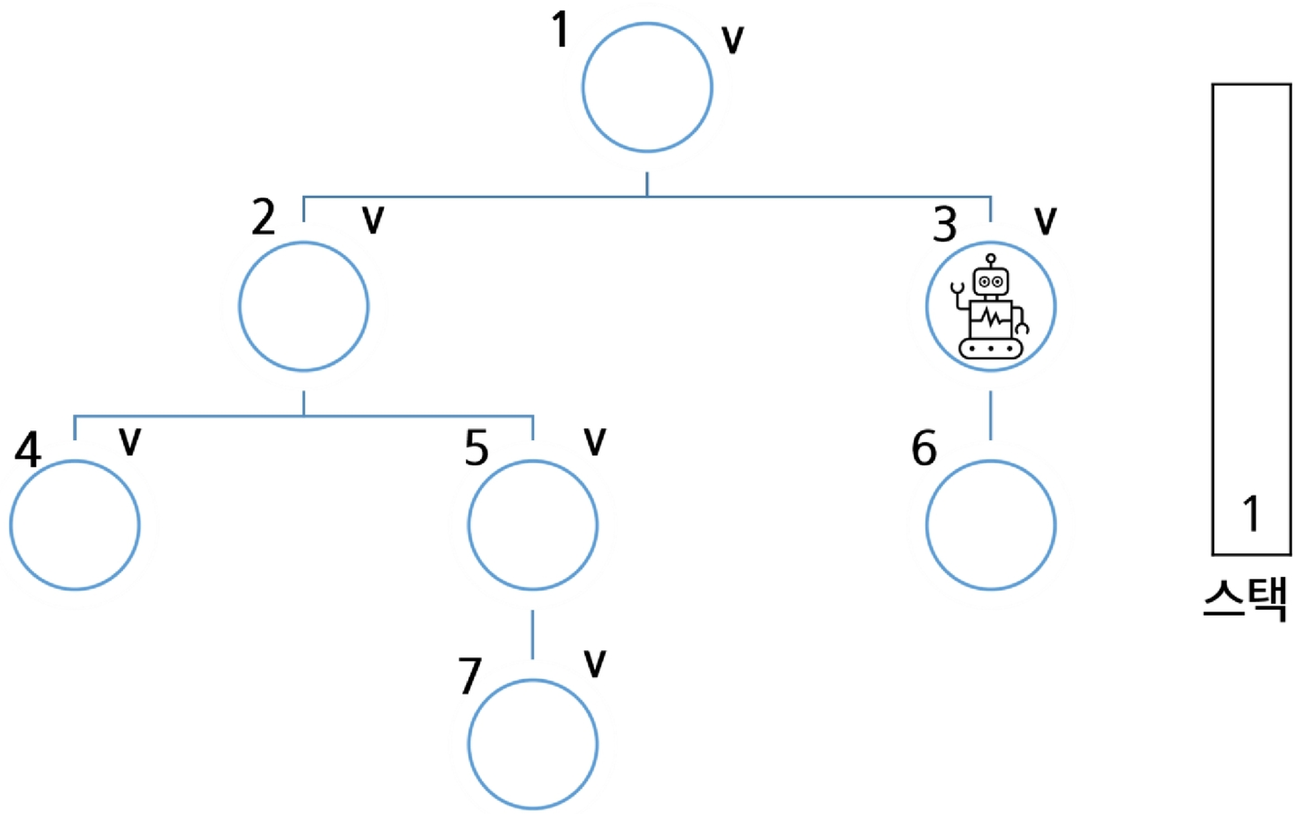
이론

로봇이 선을 따라 모든 칸을 탐색하는 방법



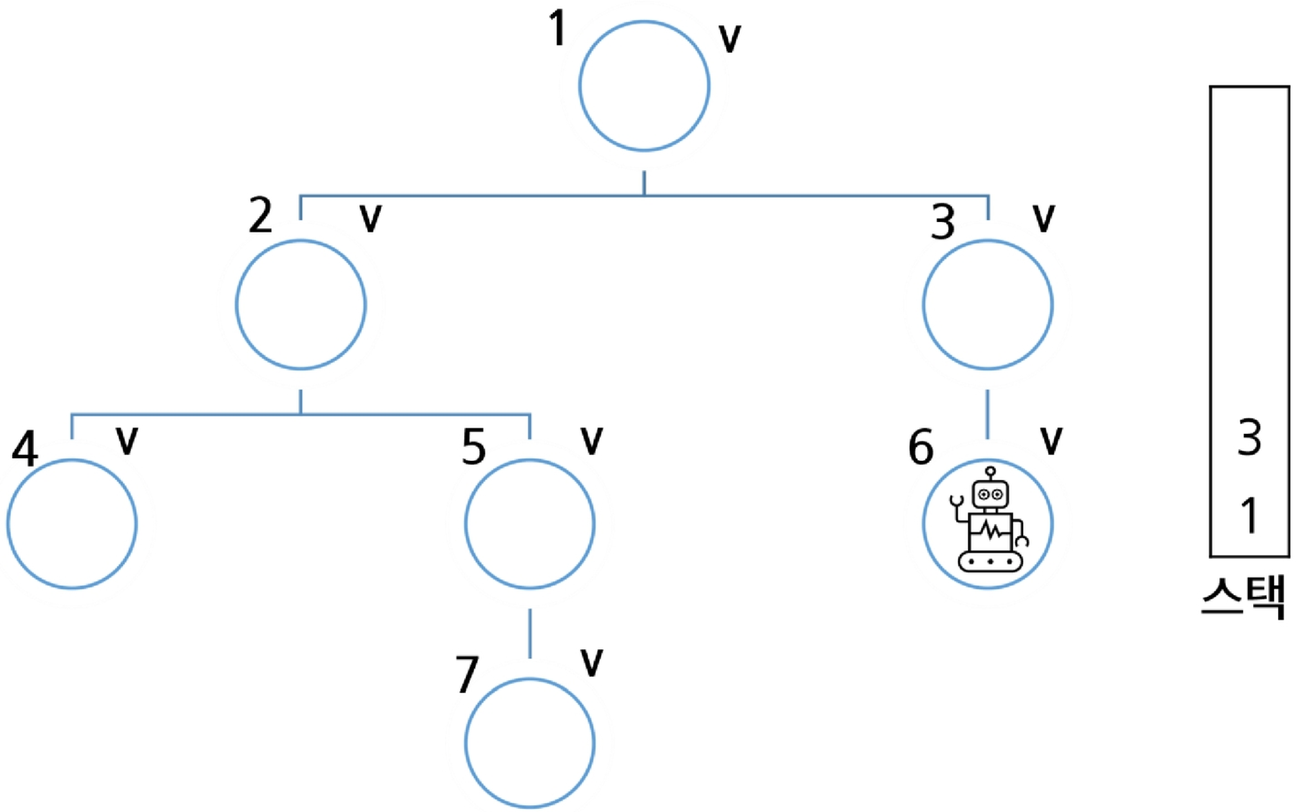
<그림24_Python_알고리즘 기본_Stack 1_로봇이 선을 따라 모든 칸을 탐색하는 방법11>

로봇이 선을 따라 모든 칸을 탐색하는 방법



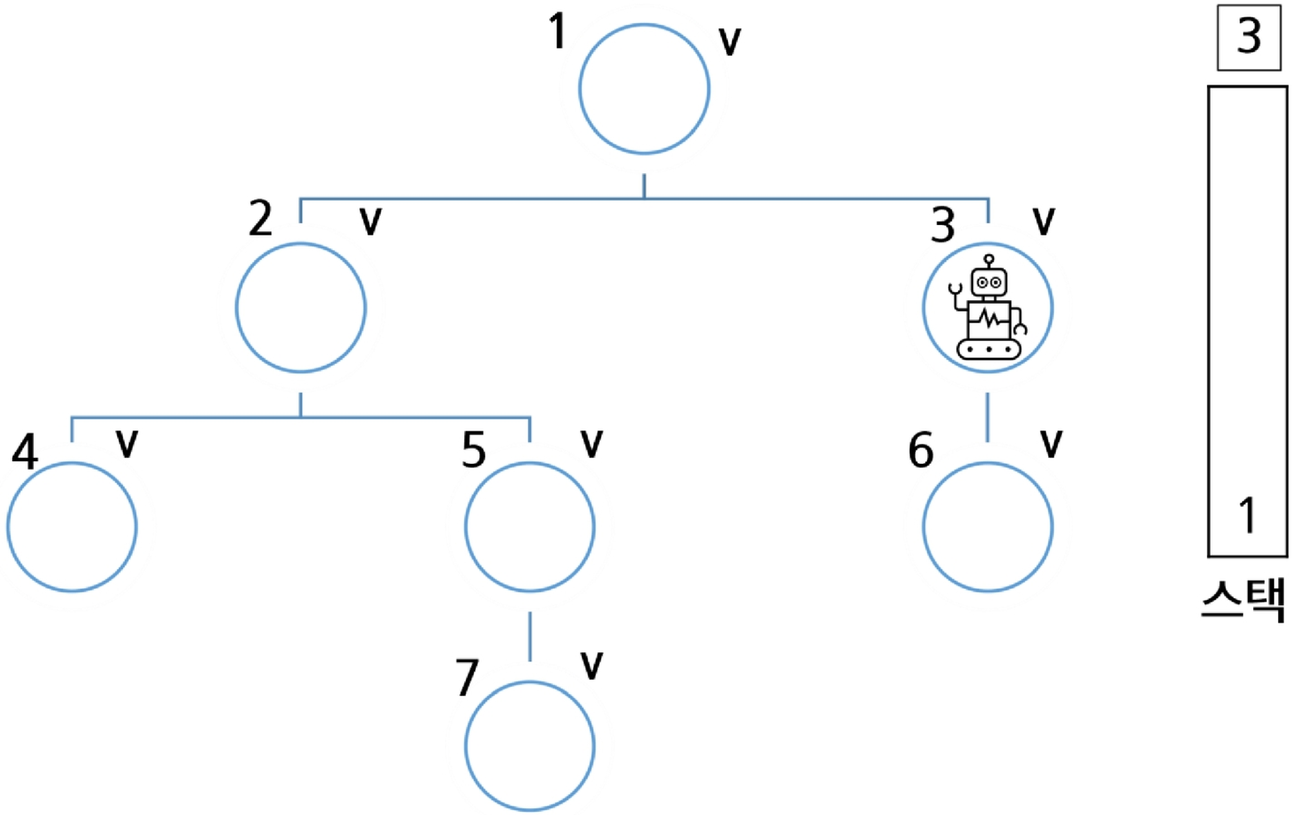
<그림25_Python_알고리즘 기본_Stack 1_로봇이 선을 따라 모든 칸을 탐색하는 방법12>

로봇이 선을 따라 모든 칸을 탐색하는 방법



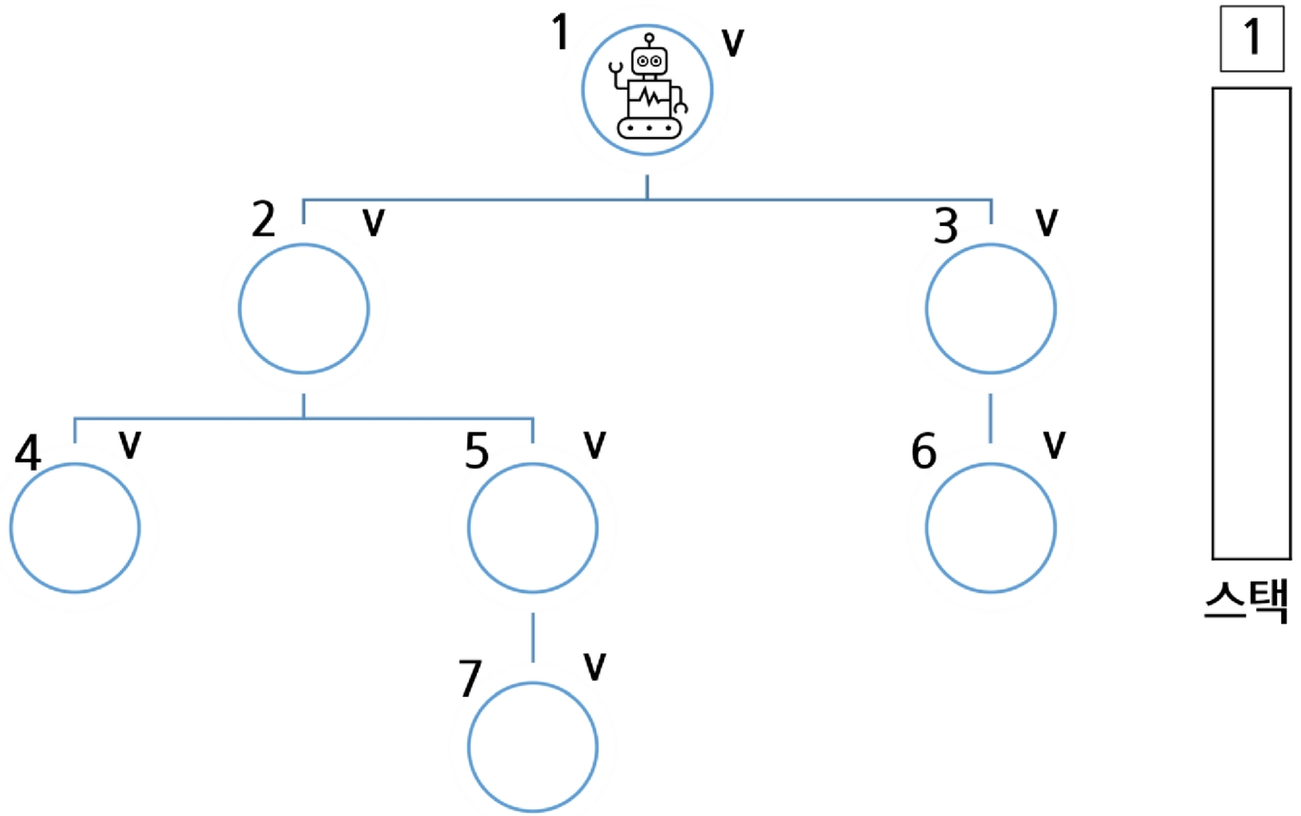
<그림26_Python_알고리즘 기본_Stack 1_로봇이 선을 따라 모든 칸을 탐색하는 방법13>

로봇이 선을 따라 모든 칸을 탐색하는 방법



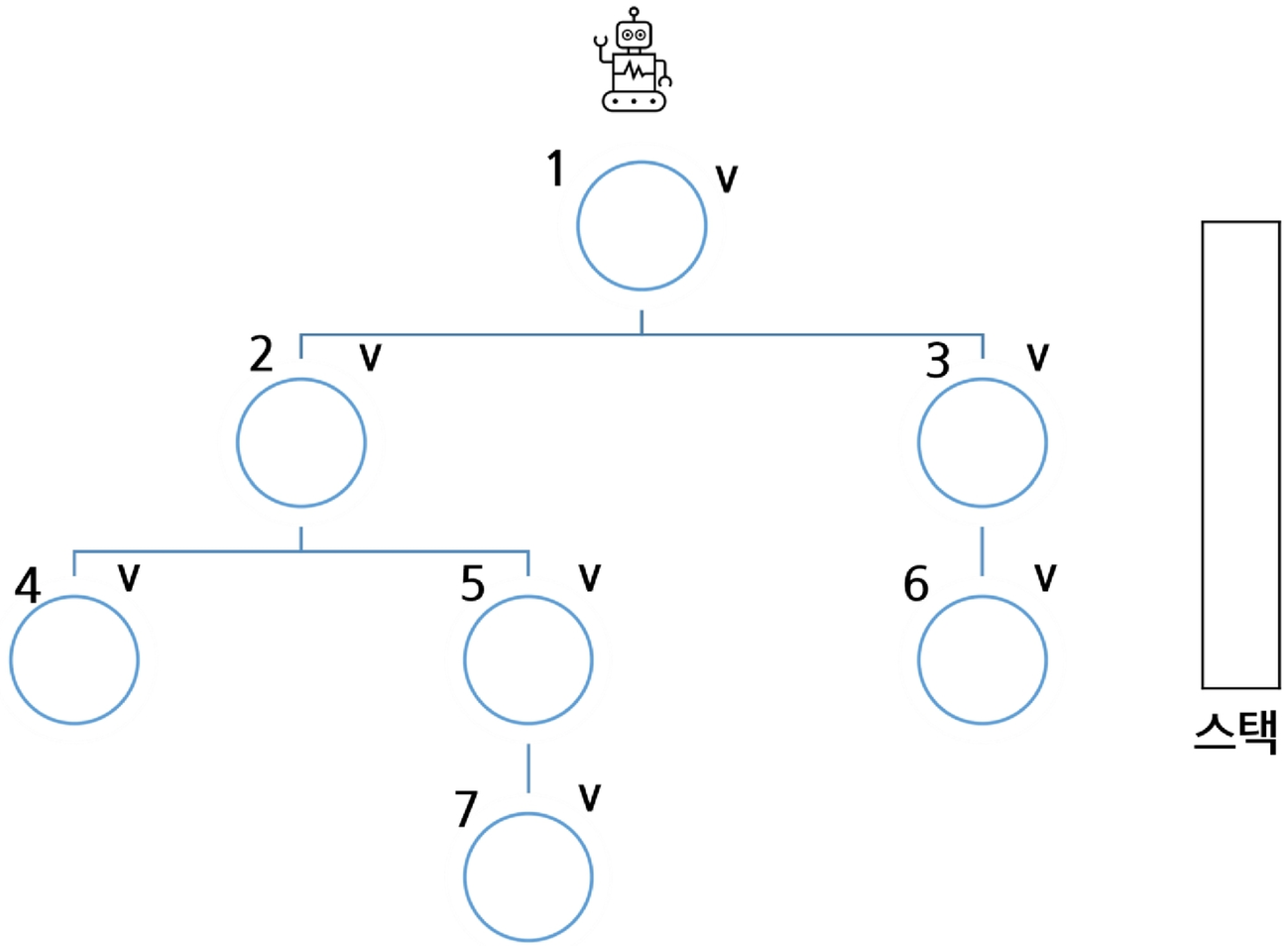
<그림27_Python_알고리즘 기본_Stack 1_로봇이 선을 따라 모든 칸을 탐색하는 방법14>

로봇이 선을 따라 모든 칸을 탐색하는 방법



<그림28_Python_알고리즘 기본_Stack 1_로봇이 선을 따라 모든 칸을 탐색하는 방법15>

로봇이 선을 따라 모든 칸을 탐색하는 방법



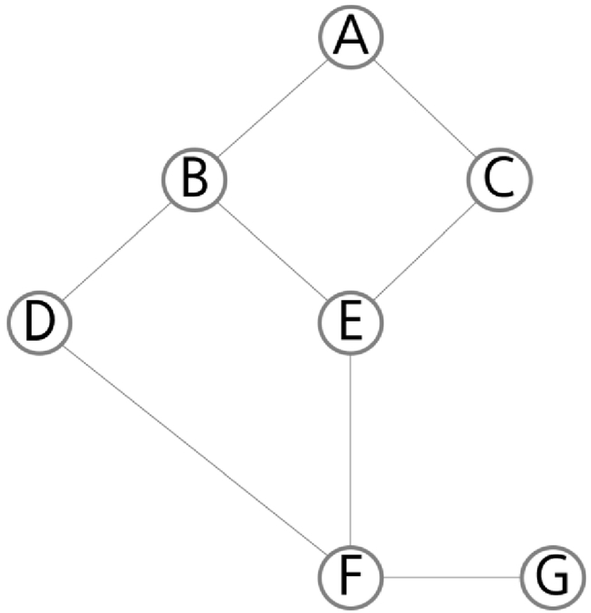
<그림29_Python_알고리즘 기본_Stack 1_로봇이 선을 따라 모든 칸을 탐색하는 방법16>

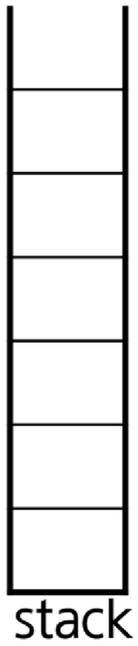
DFS 알고리즘

```
visited[], stack[] 초기화
DFS(v):
    시작점 v 방문
    visited[v]  ·  True
    while {
        if (v의 인접 정점 중 방문 안 한 정점 w가 있으면)
            push(v)
            v  ·  w    (w에 방문)
            visited[w]  ·  True
        else
            if (스택이 비어 있지 않으면)
                v <- pop(stack)
            else
                break
     }
end DFS()
```

DFS 예

초기 상태) 배열 visited를 False로 초기화하고, 공백 스택을 생성

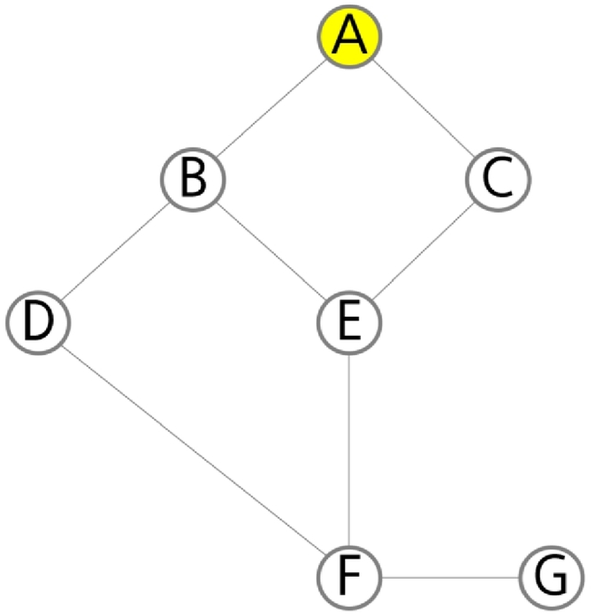


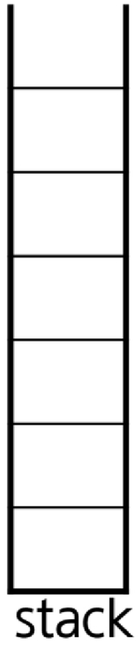


정점	A	B	C	D	E	F	G
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
visited	F	F	F	F	F	F	F

DFS 예

1) 정점 A를 시작으로 깊이 우선 탐색을 시작





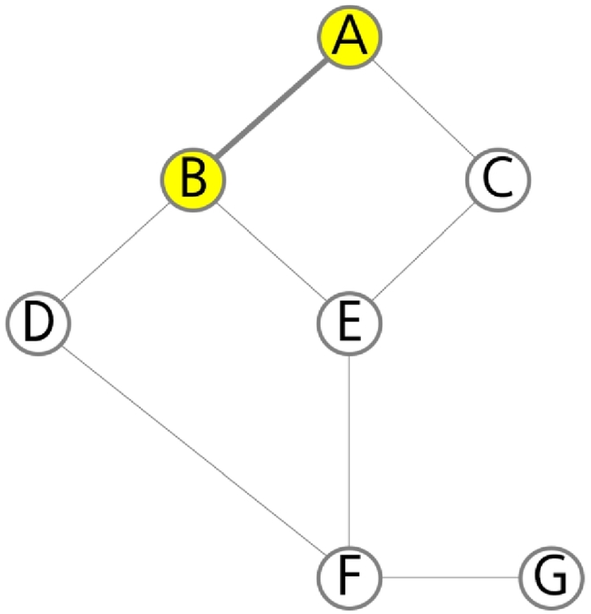
정점	A	B	C	D	E	F	G
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
visited	T	F	F	F	F	F	F

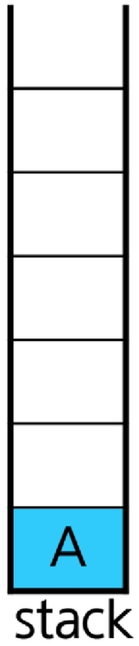
```
A 방문;
visited[A] ← True
```

<그림31_Python_알고리즘 기본_Stack 1_DFS1>

DFS 예

2) 정점 A에 방문하지 않은 인접 정점 B, C가 있으므로 A를 스택에 push 하고,
B와 C 중에서 오름차순에 따라 B를 선택하여 탐색을 계속한다.





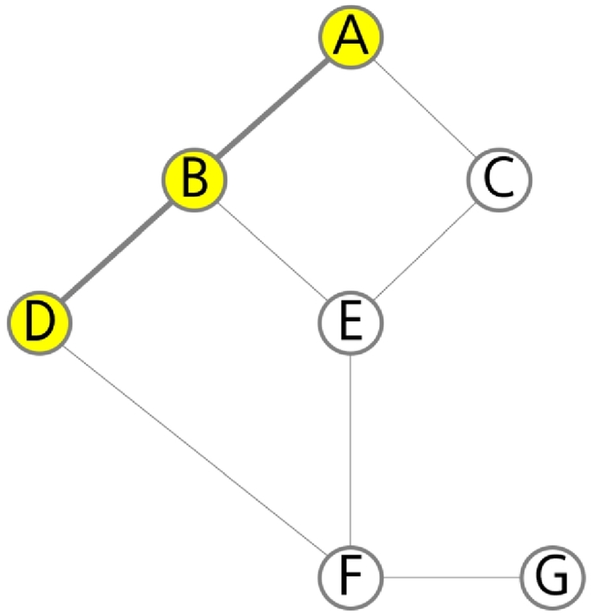
정점	A	B	C	D	E	F	G
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
visited	T	T	F	F	F	F	F

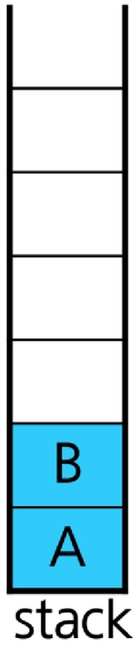
```
push(A)
B 방문
visited[B] ← True
```

<그림32_Python_알고리즘 기본_Stack 1_DFS2>

DFS 예

3) 정점 B에 방문하지 않은 인접 정점 D, E가 있으므로 B를 스택에 push 하고,
D와 E 중에서 오름차순에 따라 D를 선택하여 탐색을 계속한다.





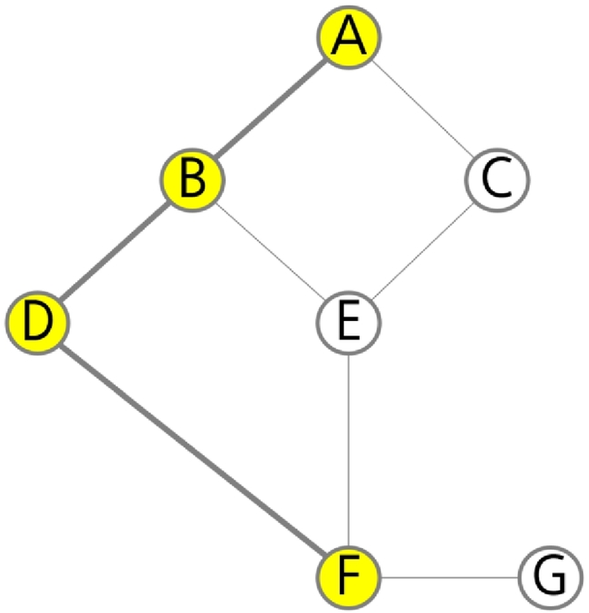
정점	A	B	C	D	E	F	G
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
visited	T	T	F	T	F	F	F

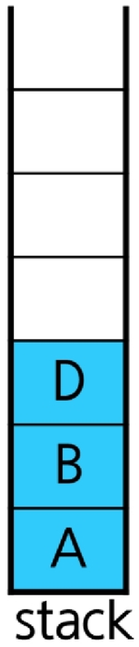
```
push(B)
D 방문
visited[B] ← True
```

<그림33_Python_알고리즘 기본_Stack 1_DFS3>

DFS 예

4) 정점 D에 방문하지 않은 인접 정점 F가 있으므로 D를 스택에 push 하고, F를 선택하여 탐색을 계속한다.





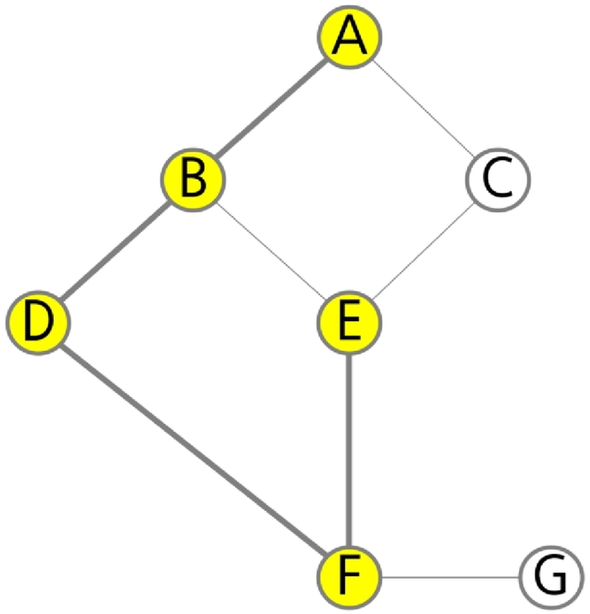
정점	A	B	C	D	E	F	G
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
visited	T	T	F	T	F	T	F

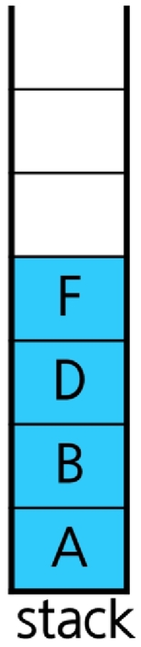
```
push(D)
F방문
visited[F] ← True
```

<그림34_Python_알고리즘 기본_Stack 1_DFS4>

DFS 예

5) 정점 F에 방문하지 않은 인접 정점 E, G가 있으므로 F를 스택에 push 하고,
E와 G 중에서 오름차순에 따라 E를 선택하여 탐색을 계속한다.





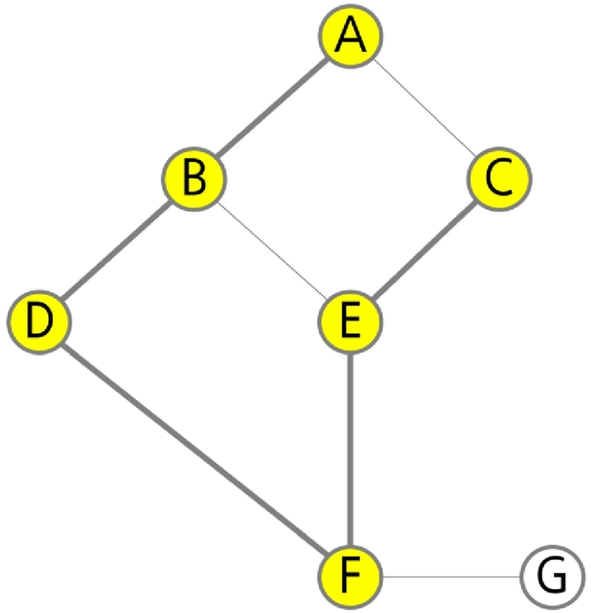
정점	A	B	C	D	E	F	G
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
visited	T	T	F	T	T	T	F

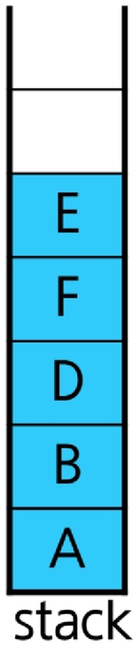
```
push(F)
E 방문
visited[E] ← True
```

<그림35_Python_알고리즘 기본_Stack 1_DFS5>

DFS 예

6) 정점 E에 방문하지 않은 인접 정점 C가 있으므로 E를 스택에 push 하고,
C를 선택하여 탐색을 계속한다.





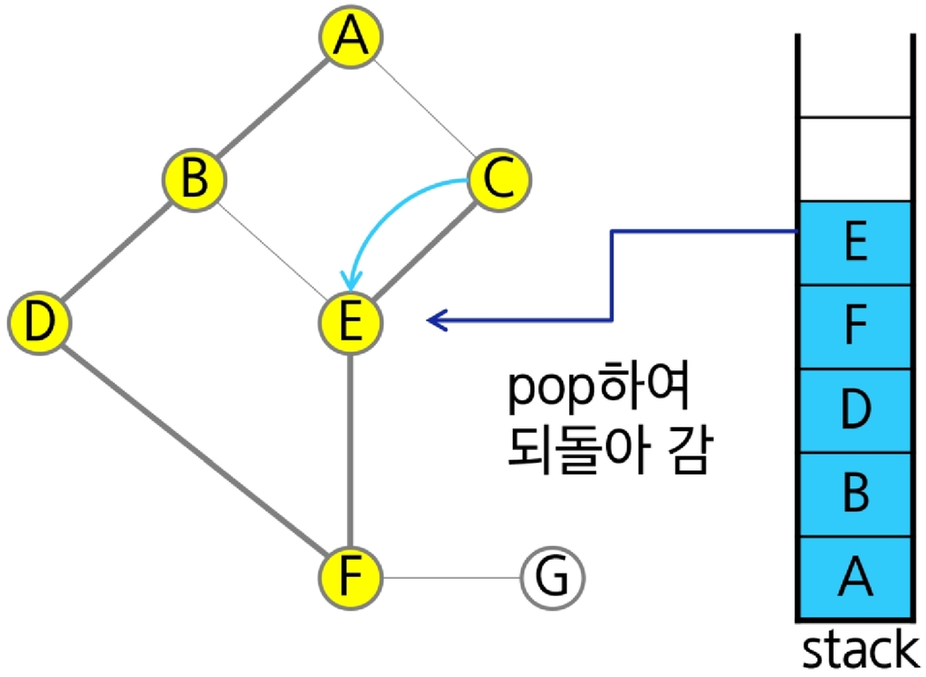
정점	A	B	C	D	E	F	G
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
visited	T	T	T	T	T	T	F

```
push(E)
C 방문
visited[C] ← True
```

<그림36_Python_알고리즘 기본_Stack 1_DFS6>

DFS 예

7) 정점 C에 방문하지 않은 인접 정점이 없으므로,
마지막 정점으로 돌아가기 위해 스택을 pop 하여 받은 정점 E에 대해서 방문하지 않은 인접 정점이 있는지 확인한다.



E의 인접 정점

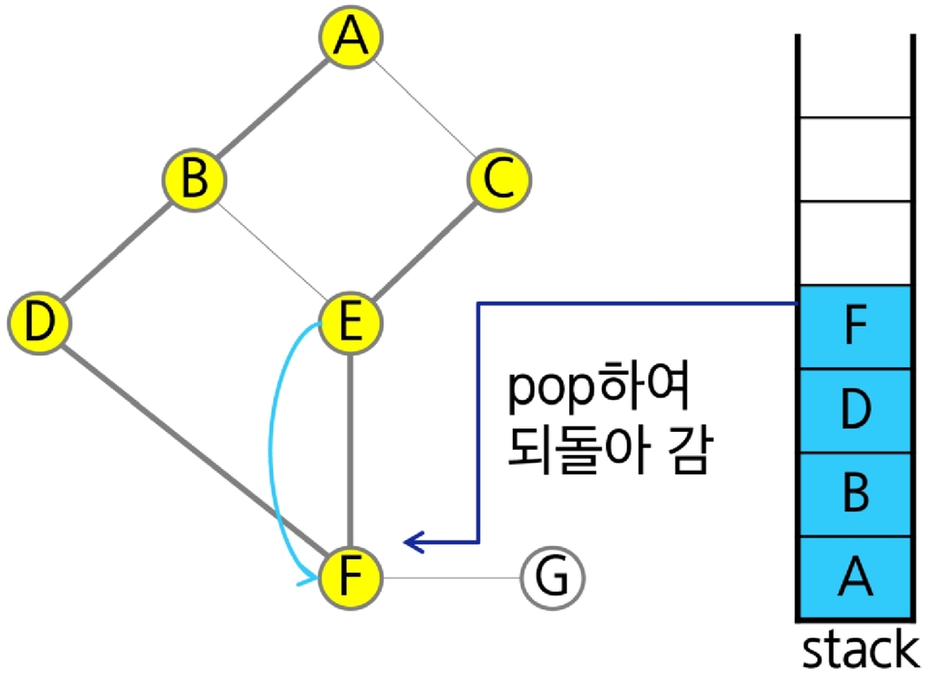
정점	A	B	C	D	E	F	G
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
visited	T	T	T	T	T	T	F

```
pop(stack)
```

<그림37_Python_알고리즘 기본_Stack 1_DFS7>

DFS 예

8) 정점 E는 방문하지 않은 인접 정점이 없으므로,
다시 스택을 pop 하여 받은 정점 F에 대해서 방문하지 않은 인접 정점이 있는지 확인한다.



F의 인접 정점

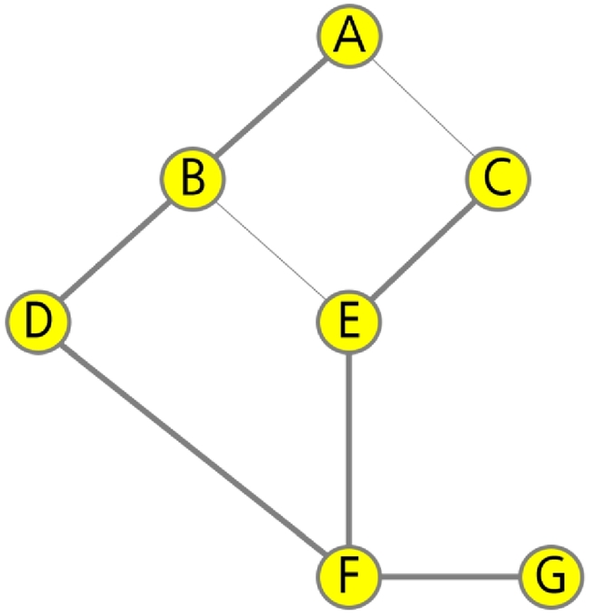
정점	A	B	C	D	E	F	G
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
visited	T	T	T	T	T	T	F

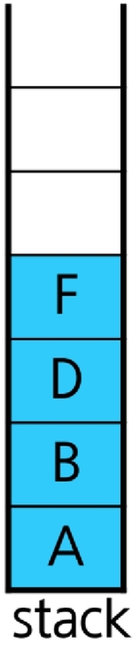
```
pop(stack)
```

<그림38_Python_알고리즘 기본_Stack 1_DFS8>

DFS 예

9) 정점 F에 방문하지 않은 인접 정점 G가 있으므로 F를 스택에 push 하고,
G를 선택하여 탐색을 계속한다.





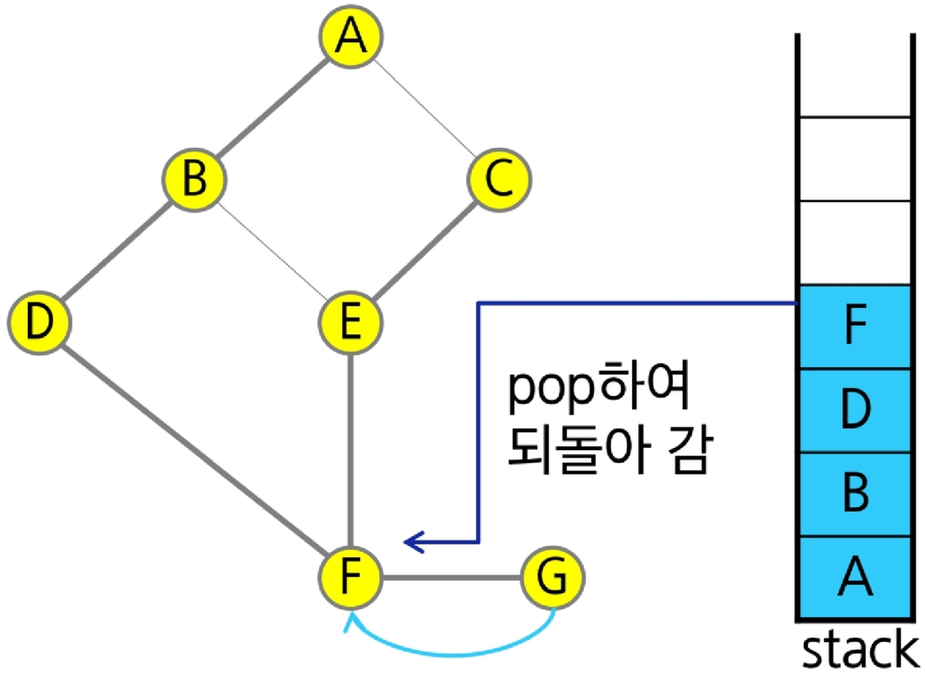
정점	A	B	C	D	E	F	G
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
visited	T	T	T	T	T	T	T

```
push(F)
G 방문
visited[G] ← True
```

<그림39_Python_알고리즘 기본_Stack 1_DFS9>

DFS 예

10) 정점 G에 방문하지 않은 인접 정점이 없으므로, 마지막 방문 정점으로 돌아가기 위해
스택을 pop 하여 받은 정점 F에 대해서 방문하지 않은 인접 정점이 있는지 확인한다.



F의 인접 정점

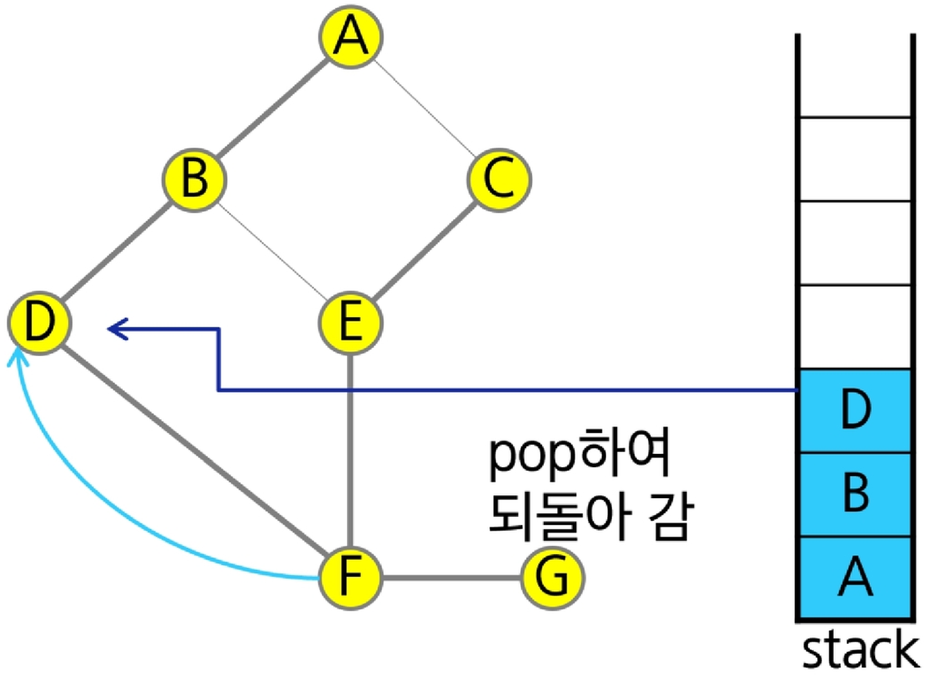
정점	A	B	C	D	E	F	G
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
visited	T	T	T	T	T	T	T

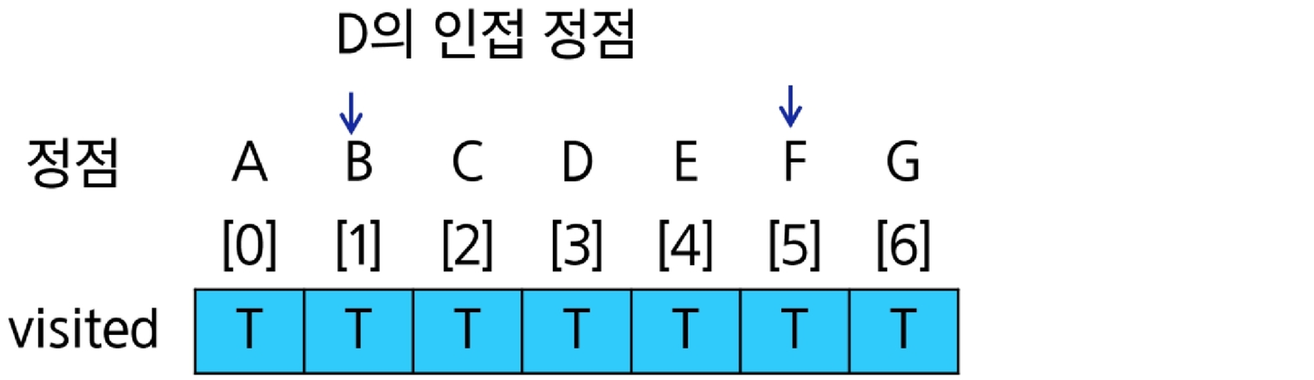
```
pop(stack)
```

<그림40_Python_알고리즘 기본_Stack 1_DFS10>

DFS 예

11) 정점 F에 방문하지 않은 인접 정점이 없으므로, 다시 마지막 정점으로 돌아가기 위해
스택을 pop 하여 받은 정점 D에 대해서 방문하지 않은 인접 정점이 있는지 확인한다.



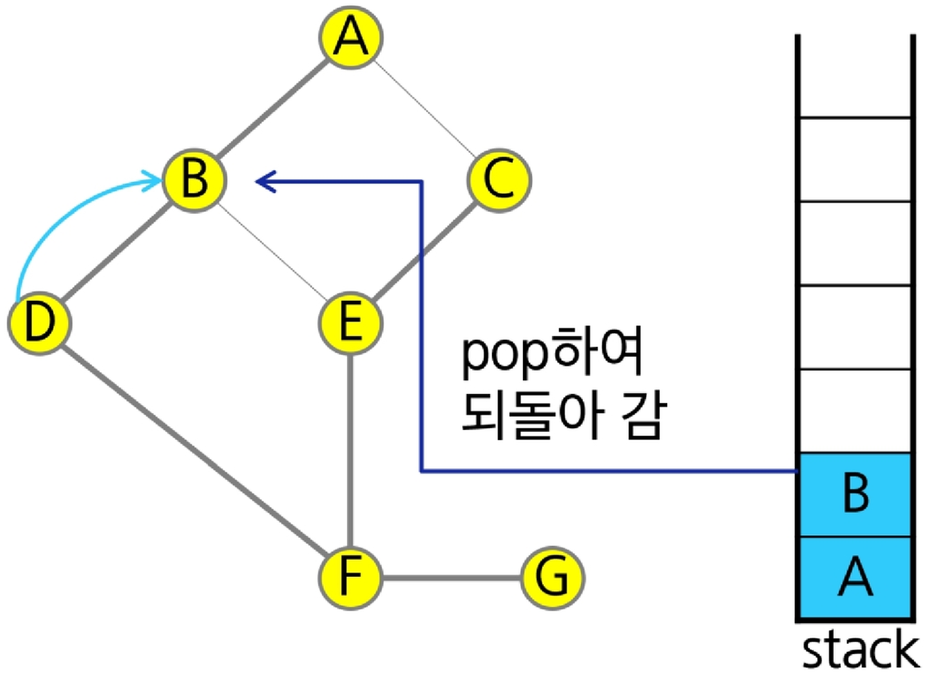



```
pop(stack)
```

<그림41_Python_알고리즘 기본_Stack 1_DFS11>

DFS 예

12) 정점 D에 방문하지 않은 인접 정점이 없으므로, 다시 마지막 정점으로 돌아가기 위해 스택을 pop 하여 받은 정점 B에 대해서 방문하지 않은 인접 정점이 있는지 확인한다.



B의 인접 정점

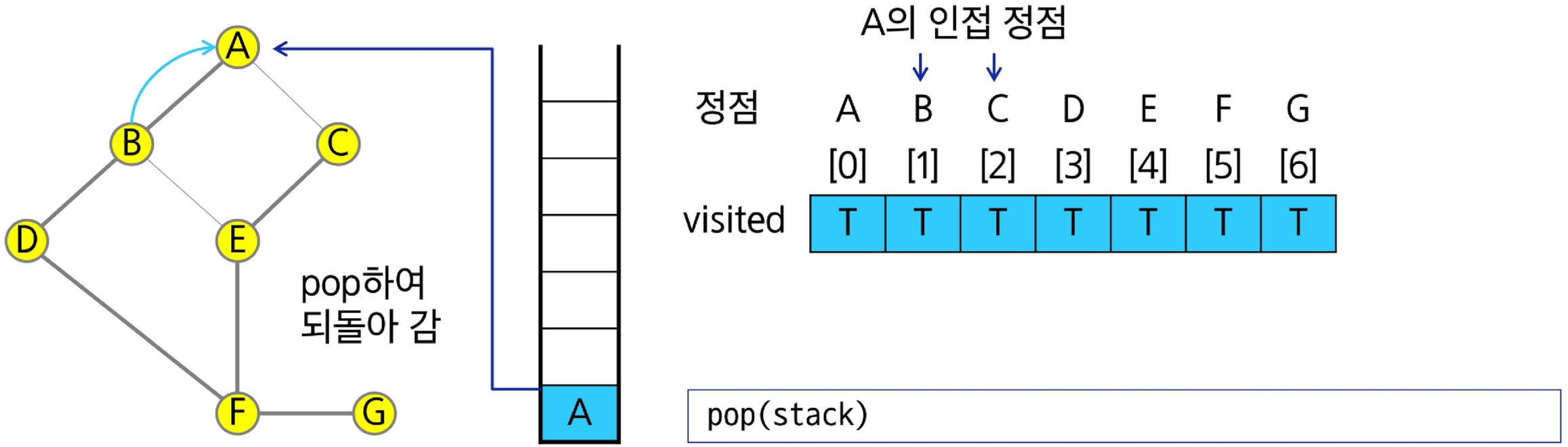
정점	A	B	C	D	E	F	G
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
visited	T	T	T	T	T	T	T

```
pop(stack)
```

<그림42_Python_알고리즘 기본_Stack 1_DFS12>

DFS 예

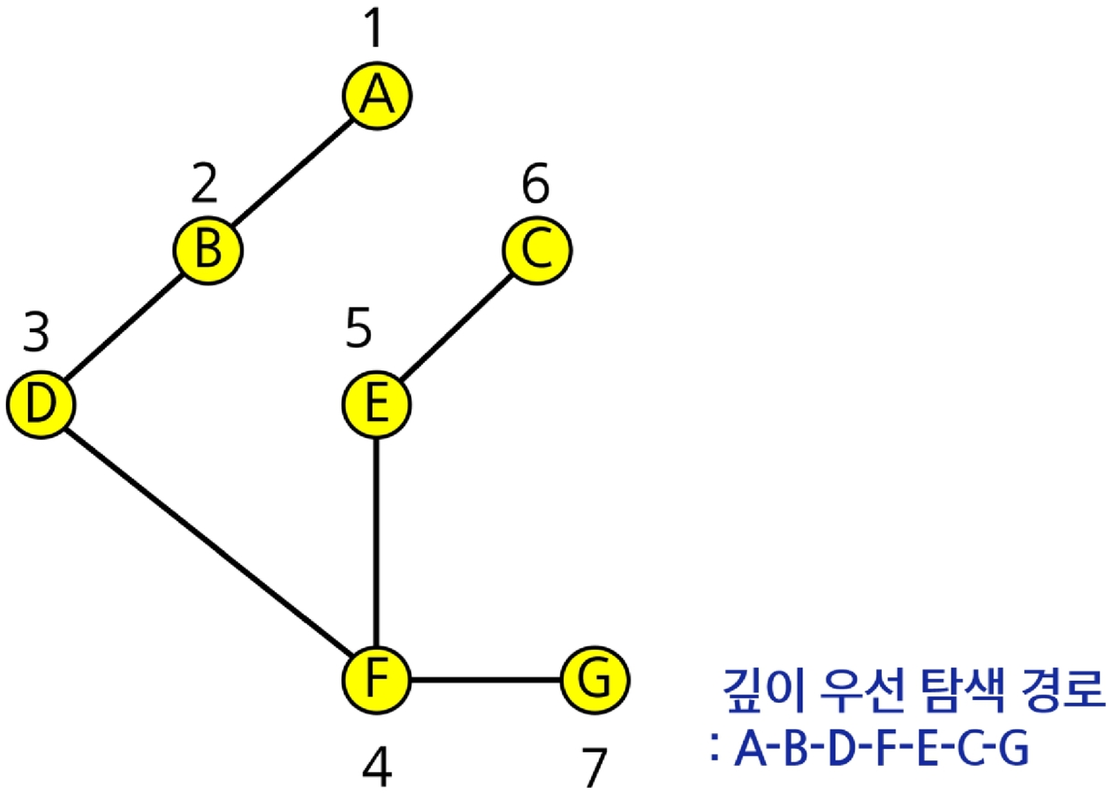
13) 정점 B에서 방문하지 않은 인접 정점이 없으므로, 다시 마지막 정점으로 돌아가기 위해 스택을 pop 하여 받은 정점 A에 대해서 방문하지 않은 인접 정점이 있는지 확인한다.



<그림43_Python_알고리즘 기본_Stack 1_DFS13>

DFS 예

14) 현재 정점 A에서 방문하지 않은 인접 정점이 없으므로 마지막 정점으로 돌아가기 위해 스택을 pop해야 하는데, 스택이 공백이므로 깊이 우선 탐색을 종료한다.

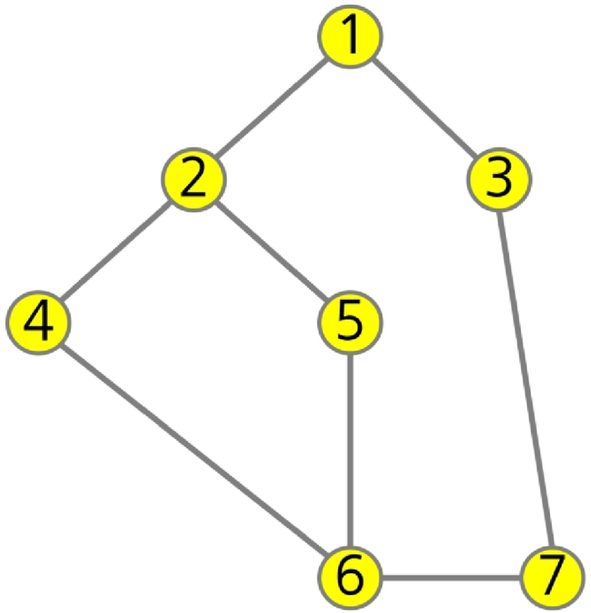


깊이 우선 탐색 경로
: A-B-D-F-E-C-G

<그림44_Python_알고리즘 기본_Stack 1_DFS14>

연습문제3

- 다음은 연결되어 있는 두 개의 정점 사이의 간선을 순서대로 나열해 놓은 것이다.
  모든 정점을 깊이 우선 탐색하여 화면에 깊이 우선 탐색 경로를 출력하시오. 시작 정점을 1로 시작하시오.




```
7 8                                  # 정점 수, 간선수
1 2 1 3 2 4 2 5 4 6 5 6 6 7 3 7      # 간선 양 끝 정점
```

출력 결과의 예는 다음과 같다
: 1 2 4 6 5 7 3

실습

Stack

- 4866. 괄호검사
- 4873. 반복문자 지우기

재귀

- 4869. 종이붙이기

DFS

- 4871. 그래프 경로

추가 연습

- 1234. 비밀번호
- 1218. 괄호 짝짓기

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 다음 질문에 답하시오. (단답형)

스택(Stack)은 어떤 데이터 삽입, 삭제 순서를 가진 선형 자료구조인가요?

2 다음 중 스택의 기본 연산에 해당하지 않는 것은?

a) push
b) pop
c) shift
d) peek

3 함수 호출과 복귀가 스택 구조로 관리되는 이유?

a) 동시성 문제 해결을 위해
b) 재귀 함수 방지를 위해
c) 가장 최근에 호출된 함수가 먼저 종료되기 때문
d) 컴파일 시간 최적화를 위해

4 다음 질문에 답하시오. (단답형)

스택이 비어 있을 때 데이터를 꺼내려고 하면 발생하는 오류 상태는?

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 배열 기반 스택 구현의 단점으로 적절한 것은?

a) 구현이 어렵다
b) 메모리 낭비가 많다
c) 스택의 크기 변경이 어렵다
d) 속도가 느리다

6 DFS 알고리즘에서 스택을 사용하는 이유는?

a) 무작위 탐색을 위해
b) 전 정점을 동시에 방문하기 위해
c) 깊은 노드를 우선적으로 탐색하기 위해
d) 가장 마지막 방문한 정점으로 되돌아가기 위해

7 다음 질문에 답하시오. (단답형)

메모이제이션(Memoization)은 어떤 방식으로 중복 계산을 줄이나요?

8 동적 계획법(DP)의 특징으로 가장 적절한 설명은?

a) 현재 선택이 항상 전체 최적 해를 보장함
b) 반복문을 사용하지 않고 재귀만을 이용
c) 동일한 부분 문제를 여러 번 계산함
d) 작은 문제의 결과를 저장하고 이를 활용해 큰 문제를 해결함

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 다음 질문에 답하시오. (단답형)

함수 호출 시 지역변수, 복귀 주소 등을 저장하는 시스템 내 저장 구조는 무엇인가요?

10 피보나치 수열을 재귀로 계산할 때 메모이제이션을 적용하면 생기는 효과는?

a) 메모리 사용량 증가
b) 중복 호출 증가
c) 실행 시간 단축
d) 정답 정확도 향상

11 피보나치 수열의 정의로 옳은 것은?

a) $F(n) = F(n-1) \times F(n-2)$
b) $F(n) = F(n-1) + F(n-2)$, 단 $F(0)=0, F(1)=1$
c) $F(n) = F(n+1) - F(n-1)$
d) $F(n) = n^2$

12 함수 호출이 스택 구조와 관련이 있는 이유는?

a) 모든 연산은 동시적으로 실행되기 때문
b) 호출된 함수의 복귀 순서를 관리하기 위해
c) 전역 변수 저장을 위해
d) 함수 정의 순서를 유지하기 위해

확인문제

정답 및 해설

1. 후입선출 (LIFO, Last-In-First-Out) 2. c) shift 3. c) 가장 최근에 호출된 함수가 먼저 종료되기 때문에 4. underflow 5. c) 스택의 크기 변경이 어렵다 6. d) 가장 마지막 방문한 정점으로 되돌아가기 위해 7. 이전 계산 결과를 메모리에 저장하여 재사용 8. d) 작은 문제의 결과를 저장하고 이를 활용해 큰 문제를 해결함 9. 스택 프레임 (stack frame) 10. c) 실행 시간 단축 11. b) F(n) = F(n-1) + F(n-2), 단 F(0) =0, F(1)=1 12. b) 호출된 함수의 복귀 순서를 관리하기 위해

1. 스택은 나중에 넣은 데이터가 먼저 나오는 구조로,
책을 쌓아올리는 방식과 유사합니다.

2. shift는 큐의 연산으로,
스택의 기본 연산인 push/pop/peek/isEmpty와는 관련이 없습니다.

3. 함수는 호출될 때마다 스택에 쌓이고,
종료 시에는 마지막에 쌓인 함수부터 차례로 제거됩니다.

4. 스택이 비어 있을 때 데이터를 꺼내려고 하면 발생하는 오류 상태입니다.

5. 배열로 구현된 스택은 고정된 크기를 가지므로
유동적인 메모리 관리가 어렵습니다.

6. DFS는 더 이상 방문할 정점이 없을 때, 마지막에 방문한 갈림길로 되돌아가
탐색을 이어가야 하므로 후입선출 구조의 스택을 사용합니다.

7. 동일한 함수 호출을 반복하지 않고,
이전 결과를 활용해 실행 시간을 줄입니다.

8. DP는 반복되는 계산을 피하고 효율적으로 문제를 해결하는
최적화 기법입니다.

9. 함수 호출 시 필요한 정보를 스택 프레임에 저장하고,
복귀 시 해당 정보를 통해 이전 상태로 돌아갑니다.

10. 중복된 연산을 피하고 결과를 재사용함으로써 효율적으로 계산할 수 있습니다.

11. 피보나치 수열은 앞의 두 항을 더해 다음 항을 만드는 수열입니다.

12. 가장 마지막에 호출된 함수가 먼저 복귀하므로,
호출 순서 관리를 위해 스택이 필요합니다.

활동정리

오늘 공부한 내용 요약 및 정리

스택(Stack)이란?

- 물건을 쌓아 올리듯 자료를 쌓아 올린 형태로, 맨 위(top)부터 꺼내는 후입선출(LIFO) 방식
- 자료의 삽입(push), 삭제(pop), 확인(peek), 공백 확인(isEmpty) 등의 기본 연산이 존재

후입선출(LIFO)

- 가장 나중에 쌓은 데이터가 가장 먼저 꺼내지는 방식

스택 응용 문제

- 괄호 검사: 여는 괄호를 스택에 저장하고, 닫는 괄호가 올 때 짝을 검사하며 처리
  모든 괄호가 짝이 맞아야 하며, 스택에 남은 괄호가 없어야 올바른 괄호 문자열
- 함수 호출 구조: 프로그램은 함수를 호출할 때마다 스택 프레임에 정보를 저장
  마지막에 호출된 함수가 먼저 복귀하며 실행 흐름을 제어

활동정리

오늘 공부한 내용 요약 및 정리

재귀호출과 메모이제이션

- 재귀 함수는 자기 자신을 다시 호출하는 구조로, 코드가 간결해짐
- 하지만 중복 호출이 많을 경우 성능 저하가 발생
- 메모이제이션(Memoization)은 이를 해결하기 위한 기법으로, 이미 계산된 값을 저장하여 중복 연산을 피할 수 있음

동적 계획법(DP)

- 작은 문제의 해답을 저장하고 이를 바탕으로 큰 문제를 해결하는 알고리즘
- 반복문 기반의 구현(iterative DP)은 재귀 + 메모이제이션 방식보다 더 효율적

DFS(깊이 우선 탐색)

- 그래프 탐색 기법으로, 한 방향으로 가능한 만큼 깊게 탐색
- 더 이상 탐색할 곳이 없으면 스택을 이용해 이전 분기점으로 되돌아가며 전체를 탐색

활동정리

핵심키워드

개념	설명	예시
스택(Stack)	데이터를 순서대로 쌓고 꺼낼 수 있는 선형 자료구조	접시를 위로 쌓고 맨 위부터 꺼내는 구조
후입선출(LIFO)	나중에 넣은 데이터가 먼저 나오는 스택의 원리	1→2→3넣으면 3→2→1순서로 꺼냄
재귀호출	함수가 자기 자신을 다시 호출하는 구조	factorial(n) = n * factorial(n-1)
메모이제이션	중복 계산을 피하기 위해 이전 결과를 저장	fibo(5) 호출 시 fibo(3) 결과 재활용
동적 계획법(DP)	작은 문제의 결과를 저장해 큰 문제를 푸는 방식	피보나치 수열에서 반복 없이 for문으로 계산
괄호 검사	스택을 사용해 괄호 짝이 맞는지 확인	(a+b)*(c-d)는 통과, (a + b))는 오류
DFS(깊이 우선 탐색)	한 정점에서 시작해 갈 수 있는 곳까지 최대한 깊게 탐색한 뒤, 더 이상 갈 곳이 없으면 이전 분기점으로 되돌아가 탐색을 이어가는 방법	미로 찾기

<표2_Python_ 알고리즘 기본_Stack 1_핵심 키워드>

활동정리

접시를 쌓아두었다가 꺼낼 때의 순서를 떠올려 봅시다.

식당에서 설거지를 끝낸 접시를 쌓는 곳이 있습니다.
아래와 같이 순서대로 접시를 올려놓습니다.

1번 접시 → 2번 접시 → 3번 접시

직원이 접시를 두 개 꺼냈다면, 꺼낸 접시 번호의 순서는?

활동정리

접시를 쌓아두었다가 꺼낼 때의 순서를 떠올려 봅시다.

식당에서 설거지를 끝낸 접시를 쌓는 곳이 있습니다.
아래와 같이 순서대로 접시를 올려놓습니다.

1번 접시 → 2번 접시 → 3번 접시

직원이 접시를 두 개 꺼냈다면, 꺼낸 접시 번호의 순서는? 3번, 2번

3번 접시 ← top 2번 접시 1번 접시	스택 : 데이터를 위로 쌓고, 맨 위(top)부터 꺼냄

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다